FIXED INCOME – Commented Study Notes

Conceptual edition from the 2026 course material

7 October 2026

This study edition explains the supplied Fixed-Income Primitives notes by concept. The substantive Notes content, figures and supplied questions are retained here, organised into conceptual chapters with additional worked explanations. The syllabus and references are retained in the appendix. The new annotated notes and the recordings of 5 October are integrated as local Prof notes; the supplied notebook and its original data are included in Part II.

Contents

I Theory **3**

1 Contracts, discounting and contractual dates **3**
- 1.1 Why Fixed-Income Primitives Matter . . . 3
- 1.2 Zero-Coupon Contracts and Spot Discount Factors . . . 3
- 1.3 Dates, Calendars, and Contractual Conventions . . . 6
- 1.4 Prof notes: Contracts, zero coupons and deterministic cash flows . . . 7

2 Spot quotations and term structures **8**
- 2.1 From Spot Discount Factors to Spot Rates . . . 8
- 2.2 The Spot Term Structure . . . 9
- 2.3 Prof notes: The same discount factor in four conventions . . . 11

3 No-arbitrage replication and forward rates **11**
- 3.1 Same Cash Flows, Same Price . . . 11
- 3.2 Price Ratios and Relative Discounting . . . 12
- 3.3 From Forward Discount Factors to Forward Rates . . . 13
- 3.4 The Forward Term Structure . . . 15
- 3.5 Prof notes: Law of one price and the forward zero coupon . . . 15
- 3.6 Mispriced forward: all cash flows . . . 16
- 3.7 Prof notes: replicate every date, not only the terminal payoff . . . 16

4 Benchmark rates: from a representation to a fixing **16**
- 4.1 Two rates enter a derivative price . . . 16
- 4.2 Fixing, accrual and payment are three different dates . . . 17
- 4.3 Compounding overnight fixings in arrears . . . 17
- 4.4 Prof notes: benchmark questions . . . 17

5 Risk-neutral pricing revisited: bank account and numeraire **18**
- 5.1 Information and the bank account . . . 18
- 5.2 Discounted prices are martingales under the pricing measure . . . 18
- 5.3 Why interest-rate claims are difficult . . . 18
- 5.4 Changing the numeraire . . . 19
- 5.5 Prof notes: the reason for changing numeraire . . . 19

6 Plain vanilla interest-rate products: a cash-flow map **19**
6.1 Linear products . . . 19
6.2 Products with optionality . . . 20
6.3 Beyond plain vanilla . . . 20
6.4 Prof notes: the pricing programme . . . 20

7 Activities with worked reasoning **20**
7.1 Supplied questions and generated study answers . . . 20
7.2 Activity 1 — Spot or forward? . . . 20
7.3 Activity 2 — One price, several rate quotations . . . 21
7.4 Activity 3 — Complete the replicating strategy . . . 21
7.5 Activity 4 — From spot prices to a forward rate . . . 22
7.6 Activity 5 — Read the term structures . . . 22
7.7 Optional AI-supported discussion . . . 22

II Code 22

8 Notebook 01: dates, conventions and market curves **23**
8.1 Learning objectives . . . 23
8.2 How we will work . . . 23
8.3 0. Setup . . . 23
8.4 Cycle 1 — Dates and calendar conventions . . . 24
8.5 Cycle 2 — Day-count and compounding conventions . . . 26
8.6 Cycle 3 — Coupon schedules . . . 29
8.7 Cycle 4 — Reading and validating a market curve . . . 30
8.8 10. Conclusions . . . 35
8.9 Homework — A convention audit . . . 35

9 Python lab: conventions, source-data checks and supporting guides **36**
9.1 Classroom map . . . 36
9.2 Reading the original LSEG data correctly . . . 36
9.3 The six supplied student exercises . . . 37
9.4 LSEG export workflow supplied for classroom verification . . . 37
9.5 NumPy array syntax . . . 38
9.6 Pandas syntax . . . 38
9.7 Jupyter: two modes and recovery . . . 39

A Course roadmap **39**
A.1 Computational applications and homework . . . 40
A.2 Assessment . . . 40
A.3 References . . . 40

Part I

Theory

1 Contracts, discounting and contractual dates

Key idea

Contract and cash-flow timeline → discount-factor price → rate quotation. A calendar controls which dates are valid; a day count controls the accrued amount. These are different inputs.

1.1 Why Fixed-Income Primitives Matter

Fixed-income securities are built from dated exchanges of money. The first task is therefore to identify the contract and its timeline; only afterwards do we choose a rate convention with which to represent its price.

This chapter is organised around a fundamental distinction between two families of contracts.

1.1.1 Spot contracts

A spot zero-coupon transaction agreed at time t exchanges an amount $P(t,T)$ at t for one unit of currency at the future date T. Its relevant interval is

$$[t,T].$$

The price $P(t,T)$ is the spot discount factor. After the dates, day-count rule, and compounding convention have been specified, the same price can be represented by a spot interest rate.

1.1.2 Forward contracts

A forward contract is agreed at time t, but the exchange to which it refers takes place at a future date. In the fixed-income application developed here, a zero-coupon bond maturing at T_2 is delivered at T_1 , with

$$t \leq T_1 < T_2.$$

The valuation date is t, whereas the relevant future interval is

$$[T_1,T_2].$$

No-arbitrage links this contract to spot zero-coupon bonds. The resulting price ratio defines a forward discount factor; only after the quotation conventions have been fixed is that price represented by a forward interest rate.

1.1.3 Prices before rates

The distinction between spot and forward is first a distinction between contracts and timelines, not between two formulas for interest rates. The logical order followed in this chapter is therefore

$$\text{contract} \longrightarrow \text{discount factor} \longrightarrow \text{rate representation}.$$

This ordering also clarifies a recurring source of error: discount factors are prices, whereas interest rates are convention-dependent representations of those prices.

1.2 Zero-Coupon Contracts and Spot Discount Factors

The basic building block of fixed-income valuation is a payment of one unit of currency at a specified future date. Its present value is represented by a zero-coupon bond price, or equivalently by a discount factor.

1.2.1 Definition

For $0 \leq t \leq T$, let

$$P(t, T) \tag{1}$$

denote the time-t value of a contract that pays one unit of currency at time T, with no intermediate payments. The date T is the maturity and T-t is the time to maturity when time is measured in years. By definition,

$$P(T, T) = 1. \tag{2}$$

For a notional amount N, the corresponding zero-coupon payment has value

$$V(t) = NP(t, T). \tag{3}$$

The notation $P(t, T)$ contains two distinct time arguments. The first, t, is the date at which the price is observed; the second, T, is the date at which the unit payment is made. When the valuation date is fixed, the map

$$T \longmapsto P(t, T), \quad T \geq t, \tag{4}$$

is called the discount function or discount curve at time t.

1.2.2 Zero-coupon bond price and discount factor

The expressions zero-coupon bond price and discount factor are often used interchangeably, but they emphasise different interpretations:

- $P(t, T)$ is a price when it refers to a traded or idealised zero-coupon security;
- $P(t, T)$ is a discount factor when it is used as the multiplier that converts a cash flow at T into a value at t.

In a modern collateralised framework, the relevant discount factors depend on the currency and collateral agreement. The notation $P(t, T)$ should therefore be understood relative to a specified discounting framework. This dependence will become explicit when curves are constructed; it is suppressed here to keep the notation simple.

1.2.3 Basic properties

In the idealised setting used in this chapter, zero-coupon payments are free of default and denominated in a fixed currency. Absence of arbitrage implies

$$P(t, T) > 0, \quad t \leq T. \tag{5}$$

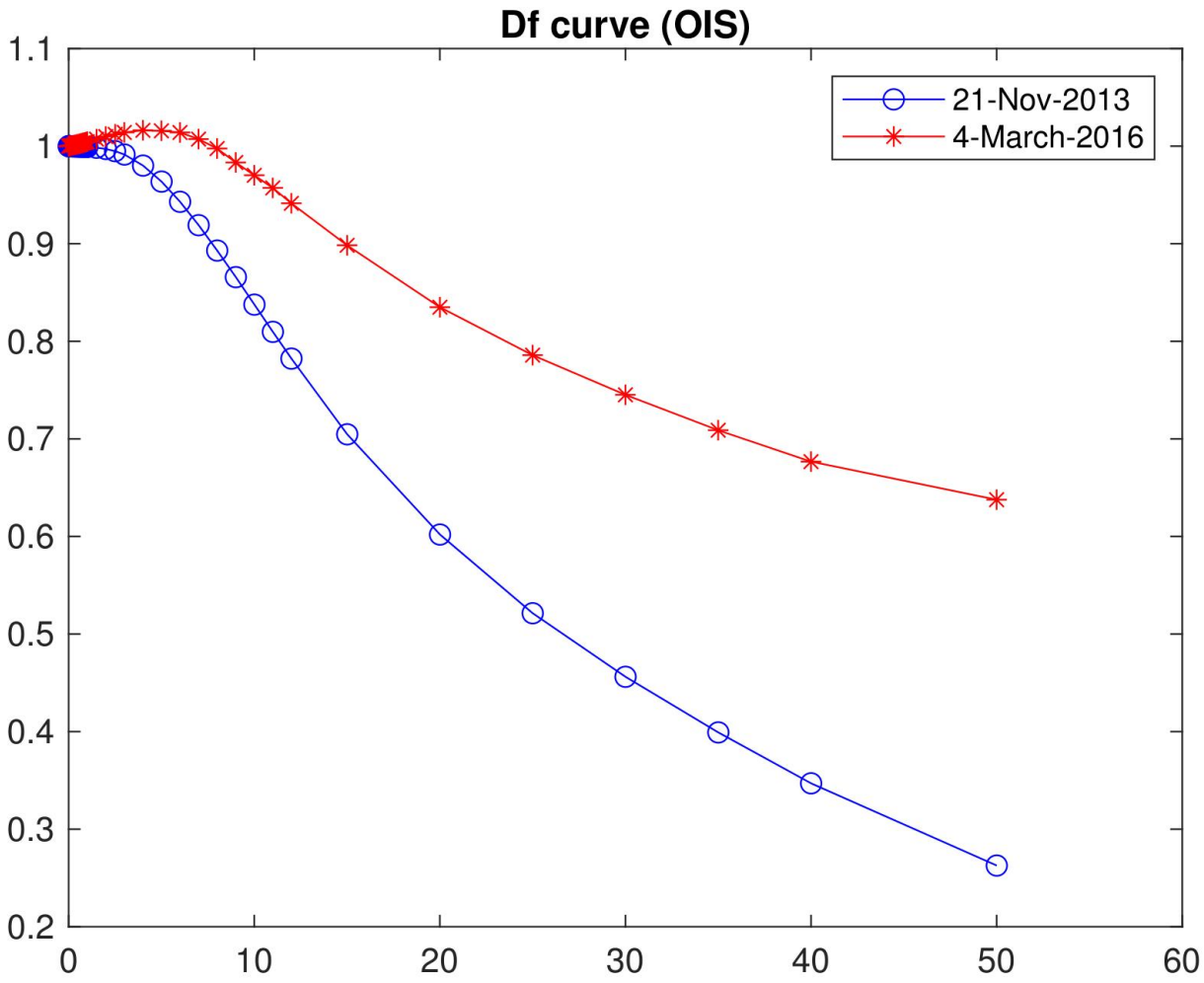


Figure 1: The discount curve at valuation date t.

A non-positive price would allow a strictly positive future payoff to be acquired without a positive initial cost.

It is common to impose

$$P(t,T) \leq 1, \quad t < T, \tag{6}$$

and to assume that $T \mapsto P(t,T)$ is non-increasing. These properties are consistent with non-negative interest rates, but they are not consequences of no arbitrage alone. If interest rates are negative over some maturities, a discount factor may exceed one and the discount curve may increase locally. The universally relevant no-arbitrage restriction in this elementary setting is positivity, not monotonicity.

1.2.4 Present and future values

If C_T is a deterministic payment at time T, its value at time t is

$$\mathrm{PV}_t(C_T) = C_T P(t,T). \tag{7}$$

Conversely, provided $P(t,T) > 0$, the time-T amount equivalent to a value $V(t)$ at time t is

$$\mathrm{FV}_T(V(t)) = \frac{V(t)}{P(t,T)}. \tag{8}$$

Thus $P(t,T)$ is the discount factor from T back to t, whereas $1/P(t,T)$ is the accumulation factor from t to T.

For deterministic cash flows C_i paid at dates T_i , value additivity gives

$$V(t) = \sum_{i=1}^{n} C_i P(t,T_i). \tag{9}$$

This formula can be interpreted as a static replication: each cash flow is replicated by C_i units of the zero-coupon bond maturing at T_i . It is the central valuation formula for deterministic fixed-income cash flows.

1.3 Dates, Calendars, and Contractual Conventions

A fixed-income contract is defined by dated cash flows. Before a rate can be computed or a cash flow discounted, calendar dates must therefore be converted into economically meaningful time intervals. This conversion is not unique: it depends on contractual and market conventions.

It is useful to keep three operations separate:

1. generate the contractual dates of the instrument;
2. adjust those dates when they are not business days;
3. compute year fractions between the relevant dates.

Changing any of these operations may change an accrual amount or a present value, even if the quoted interest rate is unchanged.

1.3.1 The relevant dates

Let t denote the valuation date. Depending on the contract, the following dates may also be required:

- the trade date, on which the transaction is agreed;
- the settlement date, on which the transaction becomes effective and the initial exchange, if any, takes place;
- the accrual start date and accrual end date, which define the interval over which interest accrues;
- the payment date, on which the corresponding cash flow is paid;
- the maturity date, which marks the contractual end of the instrument.

These labels need not identify the same calendar date. In particular, an accrual end date and its payment date may differ. The accrual factor must be computed from the dates specified by the contract, whereas discounting uses the date on which the cash flow is actually paid.

The time between the valuation date and a future payment date is also distinct from the accrual fraction of the associated interest period. Following the notation used throughout these notes, for the accrual period $[T_{i-1}, T_i]$ define its length in years as

$$\tau_i = \tau(T_{i-1}, T_i). \tag{10}$$

This accrual factor determines the interest amount. If the corresponding cash flow is paid at T_i^p , its present value uses the discount factor $P(t, T_i^p)$. Thus accrual dates determine the cash-flow amount, whereas the payment date determines which discount factor is used.

1.3.2 Business-day calendars

A business-day calendar specifies the dates on which payments and settlements can occur in the relevant market. Weekends alone are not sufficient: market holidays depend on the currency, financial centre, and contract.

Let D be an unadjusted contractual date. A business-day convention maps D into an adjusted date D^{adj} . Common conventions include:

Following	move to the first following business day;
Modified Following	move to the first following business day, unless this moves the date into the next calendar month; in that case, move to the first preceding business day;
Preceding	move to the first preceding business day;
Modified Preceding	move to the first preceding business day, unless this moves the date into the previous calendar month; in that case, move to the first following business day;
Unadjusted	leave the contractual date unchanged.

The convention is part of the contract. It should not be selected after the cash flows have been generated merely to obtain convenient dates. Moreover, the convention applied to the final maturity date may differ from the one applied to intermediate dates.

1.3.3 Day-count conventions

A day-count convention defines the year fraction

$$\tau(T_1, T_2) \tag{11}$$

associated with the calendar interval $[T_1, T_2]$. This quantity enters interest accruals and the conversion between discount factors and quoted rates. It is therefore part of the definition of the rate, not a cosmetic detail.

For example, under an Actual/360 convention,

$$\tau(T_1, T_2) = \frac{\text{actual number of days from} T_1 \text{to} T_2}{360}, \tag{12}$$

whereas under Actual/365 Fixed the denominator is 365. Conventions in the 30/360 family replace the actual calendar-day count with a rule-based count in which months are treated as having 30 days. Actual/Actual conventions use the actual number of days but determine the denominator by a convention-specific treatment of calendar years and coupon periods.

Names such as "30/360" and "Actual/Actual" identify families rather than always identifying a unique algorithm. The precise market variant must be specified whenever it affects a calculation.

1.3.4 Payment schedules

A payment schedule is an ordered sequence of dates

$$T_0 < T_1 < \cdots < T_n, \tag{13}$$

together with the conventions used to generate and adjust them. Its construction normally requires:

- an effective or settlement date and a termination date;
- a tenor, such as three or six months;
- a calendar and business-day convention;
- a date-generation rule;
- an end-of-month rule, when applicable.

Dates may be generated forward from the effective date or backward from the termination date. If the contractual horizon is not an integer multiple of the tenor, the schedule contains a short or long first or last period, called a stub. The generation direction therefore matters: it determines where an irregular period appears.

One must also distinguish between unadjusted schedule dates and adjusted payment dates. Adjusting each date independently can make consecutive accrual periods appear irregular even when the underlying contractual schedule is regular. A robust implementation should preserve both sets of dates and apply the appropriate day-count rule to the dates specified in the contract.

The practical construction of a schedule, its accrual factors, and its payment times will be developed separately in the companion notebook.

Worked example

Original pedagogical example: 90 actual days give $90/360 = 0.25$ under Actual/360 and $90/365 \approx 0.2466$ under Actual/365 Fixed. A fixed quoted rate therefore accrues different interest amounts. For a coupon $C_i = NL_i\tau_i$ paid at T_i^p, its value is $NL_i\tau_i P(t, T_i^p)$: the accrual end date need not equal the payment date.

Key idea

Before inserting any number into a rate formula, write down *which date generates the amount* and *which date receives the payment*. Keep unadjusted schedule dates, adjusted payment dates and year fractions separately.

1.4 Prof notes: Contracts, zero coupons and deterministic cash flows

Spot contracts determine spot discount factors or spot rates; forward contracts determine forward discount factors or forward rates. No-arbitrage links the forward price to spot prices through a replicating portfolio. A zero coupon has no intermediate payments. A bond costing 99 today and paying 100 in one year can be normalised to a unit bond costing $P(0,1) = 0.99$ and paying 1. Thus $P(t,T)$ is the value at time t of one monetary unit available at T. For fixed t, the discount curve is $T \mapsto P(t,T)$, $T \geq t$. No-arbitrage requires $P(t,t) = 1$ and $P(t,T) > 0$; nonnegative rates additionally imply discount factors no greater than one. The source's OIS plot shows that different valuation dates produce different curves. Discounting 200 with $P(0,1) = 0.99$ gives 198; accumulating 198 gives $198/0.99 = 200$. The accumulation factor is $1/P(0,T)$. For deterministic payments C_i at T_i, static replication with C_i units of the corresponding zero coupon yields

$$V(t) = \sum_i C_i P(t, T_i).$$

For a coupon schedule: generate contractual dates, adjust for business days, then compute accrual year fractions using the agreed day-count convention. Dates are calendar objects; $\tau_i = \tau(T_{i-1}, T_i)$ is a time interval in years. Only if the endpoints already represent compatible numerical times can it be replaced directly by a subtraction.

2 Spot quotations and term structures

2.1 From Spot Discount Factors to Spot Rates

A spot discount factor is a price. To represent that price by an interest rate, one must specify:

start date, end date, day-count convention, compounding convention. (14)

For a spot contract, the start date is the valuation date t, the end date is the maturity T, and

$$\tau = \tau(t, T) > 0 \tag{15}$$

is the corresponding year fraction. Once τ and the compounding convention have been fixed, there is a one-to-one correspondence between $P(t, T)$ and the associated spot rate.

2.1.1 Simple compounding

Let $L(t, T)$ denote the simply compounded spot rate for the interval $[t, T]$. Investing one unit at time t produces

$$1 + L(t, T)\tau(t, T) \tag{16}$$

at time T. Therefore,

$$P(t, T) = \frac{1}{1 + L(t, T)\tau(t, T)}, \tag{17}$$

and conversely

$$L(t, T) = \frac{1}{\tau(t, T)} \left(\frac{1}{P(t, T)} - 1 \right). \tag{18}$$

The admissibility condition is $1 + L(t, T)\tau(t, T) > 0$, which is equivalent to the positivity of the discount factor.

Under the no-compounding assumption, interest over a single accrual period is proportional to the length of that period. If an annualised simply compounded rate L_i applies to $[T_{i-1}, T_i]$, the interest on notional N is

$$C_i = NL_i\tau_i, \quad \tau_i = \tau(T_{i-1}, T_i). \tag{19}$$

If this amount is paid at T_i^p , its value at time t is

$$V_i(t) = NL_i\tau_i P(t, T_i^p). \tag{20}$$

The accrual dates determine the amount of interest, whereas the payment date determines the discount factor.

2.1.2 Annual compounding

Let $Y(t, T)$ denote an annually compounded spot rate. The accumulation factor over $\tau(t, T)$ years is $(1 + Y(t, T))^{\tau(t,T)}$. Hence

$$P(t, T) = (1 + Y(t, T))^{-\tau(t,T)}, \tag{21}$$

$$Y(t, T) = P(t, T)^{-1/\tau(t,T)} - 1. \tag{22}$$

2.1.3 Compounding m times per year

Let $Y_m(t, T)$ denote the nominal annual spot rate compounded m times per year. The rate applied in each compounding period is $Y_m(t, T)/m$, giving

$$P(t, T) = \left(1 + \frac{Y_m(t, T)}{m} \right)^{-m\tau(t,T)}, \tag{23}$$

$$Y_m(t, T) = m \left(P(t, T)^{-1/[m\tau(t,T)]} - 1 \right). \tag{24}$$

The quoted quantity Y_m is a nominal annual rate; it is not the rate applied in each subperiod.

2.1.4 Continuous compounding

Let $R(t,T)$ denote the continuously compounded spot rate. Then

$$P(t,T) = \exp(-R(t,T)\tau(t,T)), \tag{25}$$

$$R(t,T) = -\frac{\log P(t,T)}{\tau(t,T)}. \tag{26}$$

Continuous compounding is particularly convenient analytically because products of discount factors become sums of logarithms.

2.1.5 Equivalent rate quotations

Rates quoted under different conventions are equivalent when they generate the same discount factor for the same dates and day-count convention. Thus,

$$\frac{1}{1+L\tau} = (1+Y)^{-\tau} = \left(1+\frac{Y_m}{m}\right)^{-m\tau} = e^{-R\tau}. \tag{27}$$

The numerical rates L, Y, Y_m , and R are generally different. Comparing them without first aligning dates, day count, and compounding convention is meaningless.

For example, if $P(t,T) = 0.95$ and $\tau(t,T) = 2$, the equivalent annualised rates are approximately

Quotation convention	Equivalent rate
Simple	2.6316%
Annual	2.5978%
Semi-annual nominal	2.5812%
Continuous	2.5647%

All four quotations describe exactly the same price.

Worked example

For the source example $P = 0.95$ and $\tau = 2$, the equivalent annualised rates are: simple 2.6316%, annual 2.5978%, nominal semiannual 2.5812%, and continuous 2.5647%. No dealer is automatically cheaper because its numerical quote is lower: all four values reproduce the same price 0.95. For a \$100 maturity payment, the current value is \$95 under each convention.

Key idea

Prices are primary. If two rate quotes appear to disagree, convert both to $P(t,T)$ using the same contract dates before diagnosing an opportunity.

2.2 The Spot Term Structure

Fix a valuation date t. The family of spot discount factors

$$\{P(t,T) : T \geq t\} \tag{28}$$

defines the discount term structure at time t. Choosing a day-count and a compounding convention transforms it into the corresponding family of spot rates

$$\{R(t,T) : T \geq t\}, \tag{29}$$

usually called the zero-rate curve or spot-rate curve.

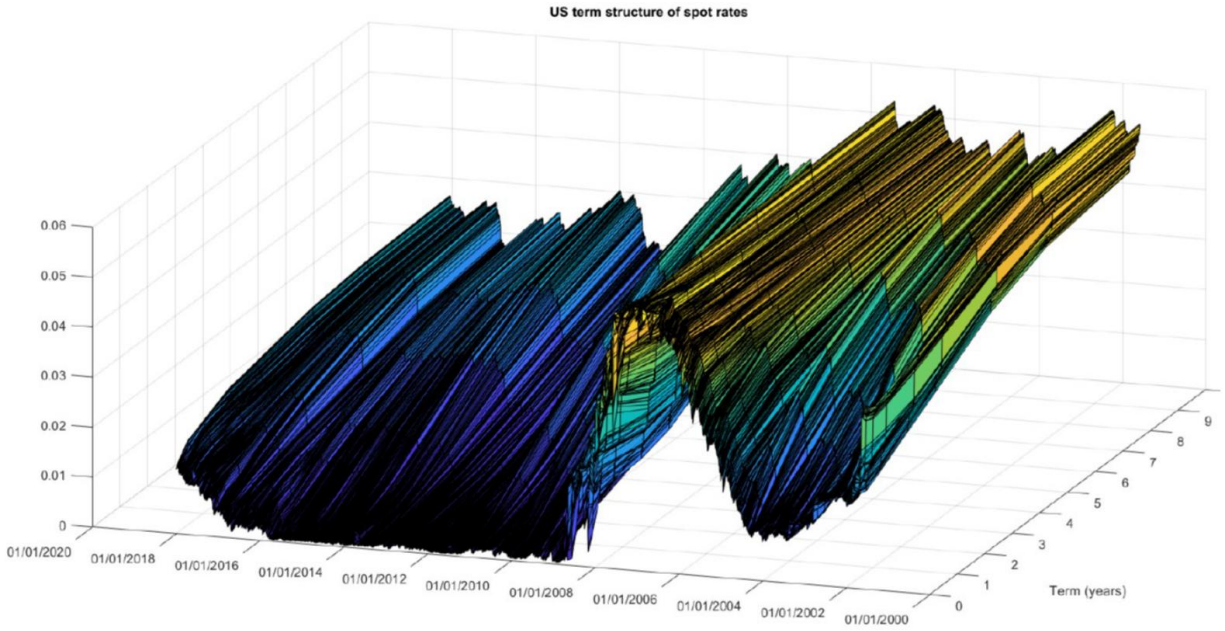


Figure 2: The term structure of spot interest rates at valuation date t.

The discount curve and a zero-rate curve contain the same information once the quotation convention is known. The discount curve is a price object and does not change when the rate convention changes. The numerical zero-rate curve does change. Consequently, two zero-rate curves should not be compared unless their day-count and compounding conventions are aligned.

2.2.1 Cross-section and evolution through time

The term structure is a cross-section across maturities at a fixed valuation date:

$$T \longmapsto P(t,T) \quad \text{or} \quad T \longmapsto R(t,T). \tag{30}$$

This should be distinguished from the evolution through calendar time of a fixed-maturity price or rate, obtained by varying t. Interest-rate models must ultimately describe this evolution, but the present chapter concentrates on the cross-sectional objects observed at one valuation date.

2.2.2 Curve shapes and terminology

A spot-rate curve may be increasing, approximately flat, decreasing, or non-monotone. The labels normal and inverted commonly refer to increasing and decreasing rate curves, respectively. These descriptions concern the selected rate representation; they should not be transferred mechanically to the discount curve. In particular, negative rates may produce discount factors greater than one and locally increasing discount curves.

The expression yield curve is sometimes used loosely for several different objects, including zero rates, par rates, and bond yields. In these notes, spot term structure means the zero-coupon term structure derived from $P(t,T)$. Par yields and yields to maturity are distinct quantities and will not be used as substitutes for spot rates.

2.2.3 Instantaneous spot rate and short rate

The continuously compounded spot rate $R(t,T)$ refers to the finite interval $[t,T]$. If the relevant limit exists, the instantaneous spot rate is defined by

$$r(t) := \lim_{T \downarrow t} R(t,T). \tag{31}$$

This quantity is called the short rate. It is the rate, observed at time t, for an infinitesimal spot contract beginning at t. When the discount curve is differentiable in maturity,

$$r(t) = -\frac{\partial}{\partial T} \log P(t,T)\Big|_{T=t}. \tag{32}$$

The notation $r(t)$ will be retained when the short rate becomes a stochastic process in the interest-rate models developed later. Unlike a rate associated with a finite market accrual period, the short rate is an infinitesimal mathematical object. It

is especially useful as the state variable in short-rate models, in which the dynamics of $r(t)$ are specified and the prices of zero-coupon bonds are then derived from those dynamics.

The general foundations of no-arbitrage pricing are developed in the Derivatives course. Here we shall use an elementary fixed-income argument based on the law of one price and replicating portfolios. Its purpose is to show how spot zero-coupon bonds price deterministic cash flows and generate forward contracts.

2.3 Prof notes: The same discount factor in four conventions

Converting a price to a rate needs valuation date, maturity date, day-count and compounding frequency. With $P(0,2) = 0.94$ and $\tau = 2$, the handwritten example derives

$$\begin{aligned} 0.94(1+2L) &= 1, & L &= \tfrac{1}{2}(1/0.94 - 1), \\ 0.94(1+Y)^2 &= 1, & Y &= (1/0.94)^{1/2} - 1, \\ 0.94(1+Y_2/2)^4 &= 1, & Y_2 &= 2[(1/0.94)^{1/4} - 1], \\ 0.94e^{2R} &= 1, & R &= -\log(0.94)/2. \end{aligned}$$

In general, $P = (1+\tau L)^{-1} = (1+Y)^{-\tau} = (1+Y_m/m)^{-m\tau} = e^{-\tau R}$, so

$$L = \frac{P^{-1}-1}{\tau}, \quad Y = P^{-1/\tau} - 1, \quad Y_m = m(P^{-1/(m\tau)} - 1), \quad R = -\frac{\log P}{\tau}.$$

Continuous compounding is the limit as the number of compounding periods tends to infinity. The curves $T \mapsto P(t,T)$, $T \mapsto R(t,T)$ and $T \mapsto Y(t,T)$ contain the same information once conventions are fixed. The supplied three-dimensional historical zero-rate plot adds a second axis, the valuation date; it is a sequence of curves, not a single curve with two maturities.

3 No-arbitrage replication and forward rates

3.1 Same Cash Flows, Same Price

Consider two feasible portfolios, A and B. Suppose that they generate exactly the same intermediate cash flows at every future payment date and the same final payoff. The law of one price implies

$$V_A(t) = V_B(t). \tag{33}$$

In words:

$$\boxed{\text{same future and intermediate cash flows } \implies \text{ same current price.}} \tag{34}$$

If, for example, $V_A(t) > V_B(t)$, an arbitrageur can sell Portfolio A and buy Portfolio B. The transaction produces the strictly positive initial cash flow

$$V_A(t) - V_B(t) > 0, \tag{35}$$

while all subsequent cash flows cancel exactly. The opposite strategy applies when $V_A(t) < V_B(t)$. Hence two portfolios with identical future cash flows cannot have different current prices in an arbitrage-free market.

This elementary argument is the fixed-income pricing principle used below: construct a portfolio of traded zero-coupon bonds that reproduces the cash flows of the target contract, and equate the two current values. The general theoretical foundations are developed in the Derivatives course.

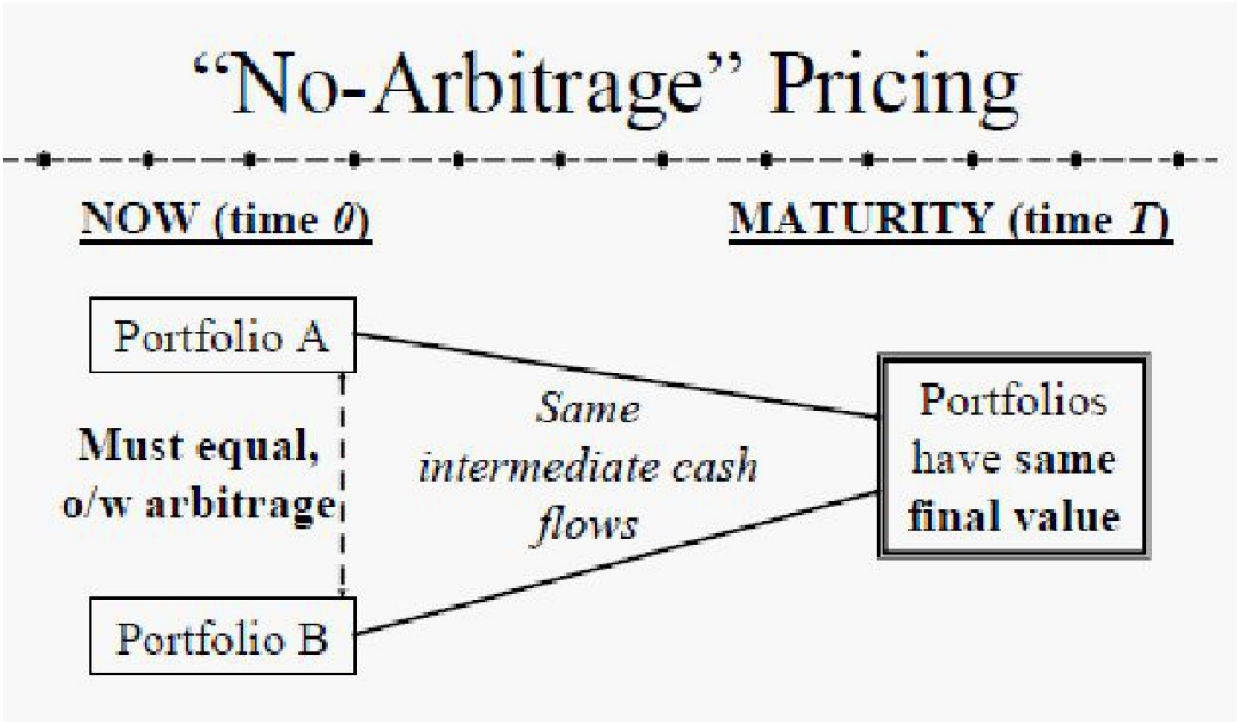


Figure 3: The Jarrow–Turnbull no-arbitrage argument: identical future and intermediate cash flows imply identical current prices.

3.1.1 Working assumptions

The elementary replication arguments in this chapter are developed under the following benchmark assumptions:

1. No market frictions: there are no transaction costs, bid–ask spreads, margin requirements, taxes, or restrictions on short sales.
2. No counterparty risk: all contractual payments used in the replication are honoured in full.
3. Competitive markets: traders are price takers and can trade at the quoted market prices.
4. No-arbitrage prices: prices adjust so that no feasible trading strategy produces an arbitrage opportunity.

These assumptions define a benchmark model. Their role is not to claim that actual markets are frictionless, but to isolate the price relations implied by replication. Frictions and counterparty or funding effects can be introduced later by modifying the relevant contracts and discounting framework.

3.2 Price Ratios and Relative Discounting

For $t \leq T_1 < T_2$, the ratio

$$\frac{P(t,T_2)}{P(t,T_1)} \tag{36}$$

compares the time-t values of unit payments at T_2 and T_1 . As will be shown below, this ratio has a direct forward interpretation. It is therefore essential to distinguish:

- $P(t,T_2)$, which discounts directly from T_2 to t;
- $P(t,T_2)/P(t,T_1)$, which discounts over the future interval $[T_1,T_2]$, as seen at time t.

3.2.1 The forward price of a zero-coupon bond

Fix three dates $t \leq T_1 < T_2$. Consider a forward contract, entered into at time t, under which k units of currency are paid at T_1 in exchange for one unit of currency at T_2 . Equivalently, the underlying asset is a zero-coupon bond maturing at T_2 , delivered at T_1 . The delivery price k is fixed at time t.

Compare the following two portfolios.

	t	T_1	T_2
Portfolio A: forward contract	0	-k	1
Portfolio B: buy one T_2-ZCB	$-P(t,T_2)$	0	1
Portfolio B: short k T_1-ZCBs	$kP(t,T_1)$	-k	0
Portfolio B: total	$-P(t,T_2)+kP(t,T_1)$	-k	1

Portfolios A and B have identical cash flows at T_1 and T_2 . Portfolio A has zero value at inception when k is the fair delivery price. By the law of one price, Portfolio B must then also have zero initial value:

$$-P(t,T_2)+kP(t,T_1)=0. \tag{37}$$

It follows that

$$k=\frac{P(t,T_2)}{P(t,T_1)}. \tag{38}$$

The quantity

$$P(t;T_1,T_2):=\frac{P(t,T_2)}{P(t,T_1)} \tag{39}$$

is the forward price, fixed at time t, of one unit paid at T_2 against payment at T_1 . It is also called the forward discount factor for the future interval $[T_1,T_2]$.

This argument makes the economic content of the formula explicit. The forward discount factor is not introduced as an algebraic ratio to be memorised: it is the unique delivery price that makes the forward contract have zero value at inception and prevents an arbitrage between the contract and its replicating portfolio.

A forward price is not, by definition, a forecast of the future spot price. It is a no-arbitrage delivery price implied by current spot discount factors. Relating it to expectations requires additional probabilistic assumptions and a specified probability measure.

3.3 From Forward Discount Factors to Forward Rates

The forward discount factor $P(t;T_1,T_2)$ is a price fixed at time t for an exchange over the future interval $[T_1,T_2]$. To represent this price by a forward interest rate, one must again specify

$$\boxed{\text{valuation date } t, \quad \text{start date } T_1, \quad \text{end date } T_2,} \tag{40}$$

The valuation date tells us when the rate is determined. The start and end dates define the interval to which the rate applies. Let

$$\tau_1=\tau(T_1,T_2)>0 \tag{41}$$

be its tenor, expressed as a year fraction. Once τ_1 and the compounding convention have been fixed, the forward discount factor and the forward rate are equivalent representations of the same no-arbitrage price.

3.3.1 Simply compounded forward rate

Let $F(t;T_1,T_2)$ denote the simply compounded forward rate, determined at time t, for the future interval $[T_1,T_2]$. It is defined by

$$P(t;T_1,T_2)=\frac{1}{1+F(t;T_1,T_2)\tau_1}. \tag{42}$$

Using the no-arbitrage expression for the forward discount factor gives

$$F(t;T_1,T_2)=\frac{1}{\tau_1}\left(\frac{1}{P(t;T_1,T_2)}-1\right) \tag{43}$$

$$=\frac{1}{\tau_1}\left(\frac{P(t,T_1)}{P(t,T_2)}-1\right). \tag{44}$$

Thus the forward rate is implied by two spot discount factors. It is not an additional independent input once the discount curve is known.

For a notional N, a simple rate $F(t;T_1,T_2)$ applied to the accrual period produces the interest amount

$$NF(t;T_1,T_2)\tau_1. \tag{45}$$

The formula concerns the amount accruing over $[T_1,T_2]$; the contract must separately specify when that amount is paid.

3.3.2 Continuously compounded forward rate

For term-structure analysis and interest-rate modelling, it is useful to represent the same forward discount factor by the continuously compounded forward rate $F_c(t;T_1,T_2)$, defined by

$$P(t;T_1,T_2) = \exp(-F_c(t;T_1,T_2)\tau_1). \tag{46}$$

Hence

$$F_c(t;T_1,T_2) = -\frac{\log P(t;T_1,T_2)}{\tau_1} = \frac{\log P(t,T_1) - \log P(t,T_2)}{\tau_1}. \tag{47}$$

The simple and continuously compounded forward rates are numerically different, but they represent the same forward discount factor:

$$\frac{1}{1+F(t;T_1,T_2)\tau_1} = \exp(-F_c(t;T_1,T_2)\tau_1). \tag{48}$$

Other compounding conventions could be defined in the same way, but they are not needed for the forward quantities used in these notes.

3.3.3 Relations between spot and forward rates

The fundamental relation is most transparent at the level of prices:

$$P(t,T_2) = P(t,T_1)P(t;T_1,T_2). \tag{49}$$

Rate relations follow by applying a common day-count and compounding convention. Under continuous compounding, write the spot rates as $R(t,T_1)$ and $R(t,T_2)$. Then

$$R(t,T_2)\tau(t,T_2) = R(t,T_1)\tau(t,T_1) + F_c(t;T_1,T_2)\tau_1. \tag{50}$$

When the year fractions are additive, $\tau(t,T_2) = \tau(t,T_1) + \tau_1$, this yields

$$F_c(t;T_1,T_2) = \frac{R(t,T_2)\tau(t,T_2) - R(t,T_1)\tau(t,T_1)}{\tau_1}. \tag{51}$$

This formula should not obscure the logical order: the price identity in Equation (49) is primary; the rate formula depends on the selected conventions.

Worked example

As an original check, let $P(t,T_1) = 0.98$, $P(t,T_2) = 0.94$, and $\delta = 0.5$. The forward discount factor is $0.94/0.98 \approx 0.959184$. The equivalent simple and continuously compounded forward quotes are respectively 8.5106% and 8.3345% per year. Substituting either quote back into its defining formula returns the same forward discount factor; their numerical difference is a convention difference, not an arbitrage.

Key idea

Draw the two dated payoffs first. A forward price is then the ratio of the two spot prices. Only after that should it be expressed as a rate.

3.4 The Forward Term Structure

At a fixed valuation date t, the collection

$$\{P(t;T_1,T_2) : t \leq T_1 < T_2\} \tag{52}$$

defines the forward discount structure implied by the spot discount curve. After a day-count and compounding convention have been chosen, it can be expressed as the corresponding collection of forward rates.

Unlike a spot curve, a forward structure has two future date arguments. A one-dimensional forward curve therefore requires one dimension to be fixed. Two common representations are:

- fix the start date T_1 and vary the end date T_2;
- fix a tenor, such as three or six months, and vary the start date T_1, with T_2 determined by the contractual date rules.

The second representation is especially natural for market forward rates. The tenor is not merely a numerical difference between two times: it is a contractual period whose end date, business-day adjustment, and year fraction must be generated using the relevant conventions.

3.4.1 Instantaneous forward rate

If the discount curve is sufficiently smooth in maturity, the continuously compounded instantaneous forward rate is defined by

$$f(t,T) = -\frac{\partial}{\partial T} \log P(t,T). \tag{53}$$

It describes the local rate in the maturity direction at T, as seen at time t. Equivalently,

$$P(t,T) = \exp\left(-\int_t^T f(t,u)du\right), \tag{54}$$

and the continuously compounded spot rate is the maturity average

$$R(t,T) = \frac{1}{\tau(t,T)} \int_t^T f(t,u)du, \tag{55}$$

when the time parameter and day-count convention are consistent with the integration variable. Thus the instantaneous forward curve contains the same information as the discount curve under the stated regularity conditions. At the short end of the curve,

$$f(t,t) = r(t), \tag{56}$$

so the short rate is the instantaneous forward rate whose start date is the current date. The instantaneous forward curve is likewise primarily a mathematical modelling object rather than a directly quoted market curve. In forward-rate models, such as the Heath–Jarrow–Morton framework, the dynamics of the family $\{f(t,T) : T \geq t\}$ are modelled directly.

3.4.2 Interpretation and curve shapes

Forward curves may be increasing, decreasing, flat, or non-monotone. Their shape need not coincide with the shape of the spot-rate curve: a spot rate is an average over an interval beginning at t, whereas a forward rate refers to a later subinterval. This difference is already visible in Equation (51).

Finally, an implied forward curve is not automatically a forecast curve. It is the set of forward prices or rates consistent with the current spot curve and no arbitrage. An expectations interpretation requires an explicit model, a probability measure, and attention to the relevant risk premia.

3.5 Prof notes: Law of one price and the forward zero coupon

The source describes arbitrage as a nonpositive-cost position with a nonnegative future payoff and a strict gain at at least one date/state. If two portfolios have identical intermediate and terminal cash flows, they must have the same current value. If both pay 100 at T, with no intermediate payments, but cost 80 and 90, buy the first and sell the second: receive 10 now and net zero at T. Buying pressure raises the cheaper price and selling pressure lowers the dearer one. The working assumptions are no market frictions, no counterparty risk, competitive markets and no remaining arbitrage opportunities. For a forward delivering a unit T_2 zero coupon at T_1, pay k at T_1 and receive 1 at T_2, with no initial payment. Given

$P(0,0.25) = 0.98$ and $P(0,0.75) = 0.956$, buy a 9M zero coupon and short k units of the 3M bond. The portfolio has the same cash flows and costs $0.956 - 0.98k$. Setting this to zero gives

$$k = \frac{0.956}{0.98} = 0.975510204\ldots.$$

The handwriting reports 0.975 as a rounded value; the unrounded ratio is used in calculations here. In general,

$$P(t;T_1,T_2) = \frac{P(t,T_2)}{P(t,T_1)}, \qquad P(t,T_2) = P(t,T_1)P(t;T_1,T_2).$$

The simple and continuous forward-rate representations are

$$F(t;T_1,T_2) = \frac{1}{\tau(T_1,T_2)}\left[\frac{P(t,T_1)}{P(t,T_2)} - 1\right], \qquad F_c(t;T_1,T_2) = -\frac{\log[P(t,T_2)/P(t,T_1)]}{\tau(T_1,T_2)}.$$

A forward loan by the lender exchanges $-N$ at T_1 for $N[1 + F\tau(T_1,T_2)]$ at T_2. The valuation date is a third date, before that future accrual period.

3.6 Mispriced forward: all cash flows

The market quotes $k = 0.95$ for the forward described above, below the no-arbitrage ratio. Buy that market forward, short one 9M zero coupon and buy 0.95 units of the 3M zero coupon:

Position	0	0.25	0.75
Long market forward	0	-0.95	1
Short 9M zero coupon	0.956	0	-1
Long 0.95 units of 3M zero coupon	$-0.95(0.98)$	0.95	0
Total	0.025	0	0

The profit is realised initially, with all subsequent obligations cancelled. The last handwritten page summarises precisely these three positions.

3.7 Prof notes: replicate every date, not only the terminal payoff

In the discussion of the forward zero coupon the lecturer initially interchanges the two price figures, then corrects them. The written example fixes the data unambiguously: $P(0,0.25) = 0.98$, $P(0,0.75) = 0.956$. Use these values rather than an isolated automatic-transcript number. The replica must match the $-k$ cash flow at three months *and* the $+1$ cash flow at nine months. Buying the nine-month zero coupon alone matches the last payment but fails at three months. Short k units of the three-month bond to produce that intermediate liability. A positive bond price is the cost of acquiring it; the buyer's initial cash flow has the opposite sign. For a mispriced market delivery price 0.95, the initial profit of 0.025 is obtained by buying the market forward, shorting one nine-month bond and buying 0.95 units of the three-month bond. Buying $0.95/0.98$ units would deliver a different amount and would no longer cancel the contractual payment. Write one cash-flow column per date and check the totals before labelling a strategy an arbitrage. The lecturer asks students to derive the simple forward relation at least once and remember it. The forward price uses longer-maturity DF divided by shorter-maturity DF; expressing its inverse as an accumulation factor reverses the ratio in the simple forward-rate formula. A forward loan is agreed today, funded at T_1 and repaid at T_2.

4 Benchmark rates: from a representation to a fixing

A rate read from a discount curve represents a price after dates, day-count and compounding have been selected. A benchmark fixing has a different contractual role: the contract names an index and a fixing date, then pays the number published for that index. The course's sequence is therefore contract, fixing, price, rate representation.

4.1 Two rates enter a derivative price

Under the course's risk-free, single-curve framework,

$$V(t) = \mathbb{E}_t^Q[D(t,T)V(T)].$$

The discount factor depends on the funding rate of the replicating strategy; the payoff of an interest-rate derivative depends on a reference rate. These are different roles even when one curve is assumed to serve both. For a European equity call the payoff is $(S_T - K)^+$; in Black–Scholes the bank account has deterministic rate r and $D(t,T) = e^{-r(T-t)}$. The rate finances the dynamic replication.

Key idea

The first part of this course assumes one curve for funding and reference rates. Keep that assumption visible: a benchmark fixing and a zero rate are not interchangeable merely because both are called interest rates.

Prof notes: risk-free price and valuation adjustments. The annotated page 4 distinguishes the risk-free price studied here from a price including counterparty and other valuation adjustments (collectively XVA). CVA concerns credit, FVA funding and KVA regulatory capital. The schematic sum in the notes is a decomposition, not a universal sign convention for all adjustments. For example, a credit-risk illustration pays 100 at year 1 if the obligor survives that date, with a possible recovery after earlier default. This illustrates credit exposure; it is distinct from the later benchmark-rate payoff discussion. Counterparty risk is excluded in the present pricing framework.

4.2 Fixing, accrual and payment are three different dates

For notional N and accrual period $[T_{i-1}, T_i]$,

$$C_i = N \,\text{Rate}_i \,\tau_i, \qquad \tau_i = \tau(T_{i-1}, T_i).$$

The fixing date determines when Rate_i becomes known; the accrual dates determine the year fraction; the payment date T_i^p determines the discount factor $P(t, T_i^p)$. The contract chooses these dates. The fixing can precede the accrual start and payment can follow its end.

	EURIBOR 6M	eSTR
Period	Six months, term rate	One business day, overnight rate
Known	Set in advance, at the start in the stylised schedule	Daily fixings; whole coupon known at the end
Coupon use	Simple rate $L(T_{i-1}, T_i)$	Compounded rate in arrears $L^{ON}(T_{i-1}, T_i)$

Prof notes: other tenors. The supplied annotations also list EURIBOR term tenors 1W, 1M, 3M, 6M and 12M. Overnight fixings can be converted into a payment over a longer period through daily compounding; they do not thereby become a fixing known at its start. In the stylised six-month example, the EURIBOR coupon is $NL(0, 0.5)\tau(0, 0.5)$, whereas the overnight coupon uses $L^{ON}(0, 0.5)$ learned over the accrual period.

4.3 Compounding overnight fixings in arrears

Let r_k be the fixing for business day k, and δ_k the year fraction for which it applies. Then

$$1 + \tau_i L^{ON}(T_{i-1}, T_i) = \prod_{k=1}^{n} (1 + r_k \delta_k), \qquad L^{ON} = \frac{\prod_k (1 + r_k \delta_k) - 1}{\tau_i}.$$

This is the simple rate equivalent to the accumulated product of daily growth factors. It is neither a geometric mean of the rates nor an arithmetic average. In particular, the Friday fixing can apply for three calendar days.

Worked example

For a Monday-to-Monday period, $\tau = 7/360$, $r = (3.00, 3.00, 3.05, 3.10, 3.10)\%$ and $\delta = (1, 1, 1, 1, 3)/360$. The growth factor is approximately 1.00059596, giving $L^{ON} = 3.0650\%$. The day-weighted average is 3.0643% and the plain average is 3.0500%. Only the compounded calculation gives the contractual coupon in this example.

The benchmark methodology and reform history are left to the source reference: Scaringi and Bianchetti (2020), *No Fear of Discounting: How to Manage the Transition from EONIA to eSTR*. This reference provides background; the pricing definitions and calculations needed above are self-contained.

4.4 Prof notes: benchmark questions

Does single curve mean a constant or flat rate? No. In the student's question around 18–20 minutes, the lecturer distinguishes using one curve for both discounting and forwarding from assuming that its rates are identical at all maturities or unchanged tomorrow. The same curve can slope and can move over time. The announced code keeps discounting and projection as separate objects even when they are initially set equal, allowing a later extension to multiple curves.

Is a six-month benchmark a six-month compounding frequency? No. EURIBOR 6M is an annualised simple fixing applied to a six-month accrual fraction. In the classroom example the start fixing is 3.5%, so for stylised $\tau = 0.5$ the coupon is $N(0.035)(0.5) = 0.0175N$. Actual contractual dates and day count determine the precise τ; a fixing or payment lag does not redefine the accrual period.

What does an overnight coupon require? The term fixing is known near the start; the overnight coupon requires the stream of daily fixings. Knowing today's overnight rate does not determine all the future factors in the product. The equivalent term payment is known only after the accrual period has been observed, subject to the contract's observation/payment conventions.

Discounting and forwarding. The funding rate enters the discount factor; the reference-rate process enters the payoff. In a single-curve setup the curve information serves both roles. When rates are stochastic the discount factor normally remains inside the expectation. The lecturer stresses translating the contract and its conventions before computing the expectation.

5 Risk-neutral pricing revisited: bank account and numeraire

5.1 Information and the bank account

The notes start from a filtered probability space $(\Omega, \mathcal{F}, (\mathcal{F}_t)_{t\geq 0}, Q_0)$, with a Brownian filtration. Q_0 denotes the historical or real-world measure, used to describe observed time series. Conditional information at time t is represented by $\mathcal{F}_t$. The bank or money-market account satisfies

$$dB_t = r_t B_t \, dt, \qquad B_0 = 1.$$

There is no dW_t term in this equation: conditional on the current short rate, the infinitesimal interest accrued over $[t, t+dt]$ is $r_t B_t dt$. This is the meaning of *locally deterministic* in the handwritten notes. If r_t is random, the bank account's future value is still random.

$$\frac{dB_t}{B_t} = r_t dt, \qquad d\log B_t = r_t dt, \qquad \log B_T - \log B_t = \int_t^T r_s ds.$$

Consequently,

$$\frac{B_T}{B_t} = \exp\left(\int_t^T r_s ds\right), \qquad D(t,T) = \frac{B_t}{B_T} = \exp\left(-\int_t^T r_s ds\right).$$

The first ratio accumulates one unit from t to T along the realised short-rate path; its inverse discounts along that path. If $r_s = r$ is constant, $D(t,T) = e^{-r(T-t)}$. The source identifies this with $P(t,T)$ in the deterministic-rate case. With stochastic rates the market zero-coupon price instead equals $\mathbb{E}_t^Q[D(t,T)]$.

5.2 Discounted prices are martingales under the pricing measure

In the no-arbitrage framework used in the lecture, there is a pricing measure Q, equivalent to Q_0, under which traded asset prices expressed in bank-account units are martingales. For an asset price N and a claim value V,

$$N(t) = \mathbb{E}_t^Q[D(t,T)N(T)], \qquad \frac{V_t}{B_t} = \mathbb{E}_t^Q\left[\frac{V_T}{B_T}\right].$$

Multiplying the second identity by B_t, which is known at time t, gives

$$\boxed{V_t = B_t \mathbb{E}_t^Q[V_T/B_T] = \mathbb{E}_t^Q[D(t,T)V_T].}$$

Key idea

Changing from Q_0 to Q changes the expectation used for pricing. It does not replace the contract's cash flow. No-arbitrage relates the price to the replicating traded assets; a historical forecast alone does not determine that price.

5.3 Why interest-rate claims are difficult

For an interest-rate derivative, V_T is itself a function of rates, for example a floating EURIBOR fixing. The pricing expectation contains both that payoff and the stochastic discount factor. Computing it requires their joint distribution under Q, rather than independent forecasts or the product of two expectations. The handwriting explicitly identifies this as the major computational issue.

5.4 Changing the numeraire

The final handwritten page generalises the bank account to a suitable strictly positive traded numeraire N. Its associated equivalent pricing measure Q^N makes prices relative to N martingales:

$$\boxed{\frac{V_t}{N_t} = \mathbb{E}_t^{Q^N}\left[\frac{V_T}{N_T}\right].}$$

Choosing $N = B$ recovers the previous equation. A different numeraire requires its associated measure; one cannot keep the same expectation measure while arbitrarily changing the denominator. These statements use the course's idealised no-arbitrage assumptions and eligible traded numeraires.

5.5 Prof notes: the reason for changing numeraire

The lecturer reviews the bank account, its logarithm and the stochastic discount factor before changing measure. Historical probability is the real-world measure (commonly denoted P); the manuscript uses Q_0, while Q denotes bank-account risk-neutral pricing. Keep the roles distinct even when notation differs across previous courses. Being locally deterministic does not mean that B_T/B_t is already known at t: the future short-rate path enters its integral. The bank account is adapted, and its current value B_t can be taken out of a conditional expectation. Its future value B_T generally cannot. For an interest-rate payoff, both $D(t,T)$ and the payoff depend on interest rates. Their joint distribution creates a computational issue. A unit zero coupon is the special simple case $V_T = 1$, giving $P(t,T) = \mathbb{E}_t^Q[D(t,T)]$. More generally, replacing the expectation of a product by the product of expectations needs a suitable conditional factorisation assumption; calling two quantities uncorrelated without specifying the conditioning is insufficient. The choice of numeraire is opportunistic: choose a traded asset that simplifies the denominator. The lecturer previews the maturity-T zero coupon $N_t = P(t,T)$ and its associated forward measure Q^T. Since $P(T,T) = 1$, the general numeraire formula becomes

$$V_t = P(t,T)\mathbb{E}_t^{Q^T}[V_T].$$

This is a preview of the next derivation, not a claim that the recording proves every measure-change step. It changes the measure as well as the numeraire, avoiding the bank-account discount factor inside the final expectation.

6 Plain vanilla interest-rate products: a cash-flow map

Let N be notional, $T_0 < \cdots < T_n$ a schedule, $\tau_i = \tau(T_{i-1}, T_i)$ and $L_i = L(T_{i-1}, T_i)$ the fixing set at T_{i-1}. L_i^{ON} is the overnight rate compounded in arrears and K is a fixed strike. Cash flows below are paid at the accrual end unless specified otherwise. They describe contract payoffs, not present prices.

6.1 Linear products

Zero-coupon and fixed-coupon bonds.
A zero coupon pays N at T. A coupon bond pays $Nc\tau_i$ at each T_i and principal N at T_n.

Floating-rate note.
Coupons $N\tau_i L_i$ at each T_i, plus N at T_n; the index can instead be compounded eSTR.

Deposit.
A simple-rate loan: $-N$ at T_0 and $N[1 + \tau(T_0,T)L(T_0,T)]$ at T.

Forward rate agreement.
The source uses end-period settlement: a fixed-rate receiver obtains $N\tau(T_1,T_2)[K - L(T_1,T_2)]$ at T_2. Keep this convention separate from an FRA settled and discounted at T_1.

Interest-rate swap.
For a fixed-rate payer, each T_i payment is $N\tau_i(L_i - K)$. An IRS is a portfolio of period exchanges with the same fixed rate. The source uses 6M EURIBOR as a floating index.

Overnight indexed swap.
The same fixed-versus-floating exchange with compounded overnight coupons: $N\tau_i(L_i^{ON} - K)$ at T_i; in euro the index is eSTR.

Interest-rate future.
The source's long position has cumulative rate payoff $N\tau(T_1,T_2)[R_0 - L(T_1,T_2)]$, delivered through daily margining. Margining creates a convexity adjustment relative to a forward; linearity of the terminal rate exposure does not make the two prices identical.

Basis swap.
Exchanges floating indices or tenors. In the stylised aligned-period example, the payment is $N\tau_i[L_i^{3M} + s - L_i^{6M}]$, with spread s on the 3M leg. In the ideal single-curve framework the fair spread is zero. The supplied material notes that basis swaps existed before 2007 with near-zero quoted spreads; materially nonzero spreads after the credit crunch indicate the need for multiple curves. Actual tenor schedules need not be aligned.

6.2 Products with optionality

With $x^+ = \max(x, 0)$, a caplet pays $N\tau_i(L_i - K)^+$ and a floorlet pays $N\tau_i(K - L_i)^+$ at T_i. A cap or floor is a portfolio of these options over consecutive periods:

$$\text{Cap}: \sum_i N\tau_i(L_i - K)^+, \qquad \text{Floor}: \sum_i N\tau_i(K - L_i)^+.$$

A long cap and short floor form a collar (the opposite positions give its reverse):

$$\sum_i N\tau_i\big[(L_i - K_c)^+ - (K_f - L_i)^+\big], \qquad K_f < K_c.$$

A European swaption gives the right to enter an IRS at T_0. Writing

$$A(T_0) = \sum_{i=1}^{n} \tau_i P(T_0, T_i),$$

for the annuity, and $S(T_0)$ for the underlying swap rate, the source's cash-settled value at exercise is $NA(T_0)(S(T_0) - K)^+$ for a payer and $NA(T_0)(K - S(T_0))^+$ for a receiver. A physically exercised swaption enters the corresponding swap rather than paying that number as an immediate coupon. A call on a unit zero coupon maturing at $S > T$ has payoff $(P(T, S) - K)^+$ at T. The supplied notes connect caps to options on zero coupons and swaptions to options on coupon bonds. A further decomposition of a coupon-bond option into zero-coupon options requires the appropriate model conditions; it is not a general payoff identity in arbitrary multi-factor models.

6.3 Beyond plain vanilla

The source lists Bermudan and American swaptions, constant-maturity swaps (CMS) and options on CMS, range accruals, callable bonds, rate digitals and options on rate spreads. These identify later applications; the present material does not supply a full pricing model for every item.

6.4 Prof notes: the pricing programme

The lecturer distinguishes linear products from optional payoffs. Fixed coupons, floating coupons, FRAs and swaps can first be understood through static cash-flow replication; an IRS collects period exchanges. Optional products require a dynamic/model-based argument because the payoff reacts nonlinearly to rate evolution. The expectation is the mathematical representation of a pricing strategy, not an unrelated forecast. The announced Python work progresses from bonds and floating notes to FRA/swap pricing and then rate options. The lecturer proposes comparing model-based analytical prices with numerical conditional expectations, including Monte Carlo, with models calibrated to market observations. Some spoken model names are not reliably recognised; the recording does not yet supply all the future code or formulas. The product sheet serves as a reference map, and the lecturer does not promise to cover every listed exotic product in this session.

7 Activities with worked reasoning

7.1 Supplied questions and generated study answers

These activities can be discussed in pairs before a class comparison. The aim is to identify the contract and its timeline before selecting a formula or a rate convention.

The questions below are retained from Notes. Each following answer is an original study aid, not a supplied model solution.

7.2 Activity 1 — Spot or forward?

For each object below, identify the valuation date, the start date and the end date of the relevant interval. Then classify it as a price or a convention-dependent rate representation:

$$P(t,T), \qquad R(t,T), \qquad P(t;T_1,T_2), \qquad F(t;T_1,T_2), \qquad f(t,T).$$

Draw the timeline for each forward object. Explain why the distinction between spot and forward concerns the underlying contracts before it concerns their rate quotations.

Generated study answer

Expression	Interpretation
$P(t,T)$	Spot price at t of one unit at T.
$R(t,T)$	Spot/zero rate from t to T, under a stated convention.
$P(t,T_2)/P(t,T_1)$	Forward discount factor for $[T_1,T_2]$.
$F_s(t;T_1,T_2)$	Simple forward rate for that interval.
$f(t,T)$	Instantaneous forward rate at maturity T.

All are assessed at t. For the spot objects the relevant interval starts at t and ends at T. For each finite forward object the interval starts at T_1 and ends at T_2; for $f(t,T)$ it is the infinitesimal interval at T. None is a realised future interest rate merely because it contains a future date.

7.3 Activity 2 — One price, several rate quotations

A discount factor $P(t,T)$ and the dates t,T are given.

1. List the additional information required to convert $P(t,T)$ into a numerical spot rate.
2. Two dealers quote different numerical rates for the same dates. Give conditions under which both quotations may nevertheless represent the same price.
3. A student concludes that the dealer quoting the lower rate offers the cheaper zero-coupon contract. Explain why the conclusion cannot be drawn from the rates alone.
4. State which object remains unchanged when the compounding convention is changed.

Generated study answer

Before computing a rate, specify the start and end dates, day-count/year fraction, and compounding convention. For a given P and τ, then solve $L=(P^{-1}-1)/\tau$, $Y=P^{-1/\tau}-1$, or $R=-\log P/\tau$ according to that convention. To compare two dealer quotes, reconstruct the discount factor from each on identical contractual terms. Equal discount factors mean equal economic value even if the percentages differ; the lower numerical rate is not necessarily the cheaper contract. The dated price $P(t,T)$ remains unchanged when only the quotation convention changes. The conventions in Section 2 supply all four conversion pairs.

7.4 Activity 3 — Complete the replicating strategy

A forward contract entered into at t exchanges k units at T_1 for one unit at T_2 , where $t<T_1<T_2$. Without consulting the derivation in the text:

1. construct the replicating portfolio using T_1 - and T_2 -zero-coupon bonds;
2. prepare a cash-flow table with columns t, T_1, T_2 ;
3. determine the value of k that gives both portfolios the same current price;
4. if the quoted delivery price is higher than this value, specify which portfolio should be bought and which should be sold. Verify every cash flow of the resulting arbitrage strategy.

Identify which working assumptions are needed for the argument.

Generated study answer

An at-market forward to pay K at T_1 and receive one unit at T_2 has future vector $(-K,+1)$. The bond portfolio long one T_2-bond and short K T_1-bonds matches this vector exactly:

Position	t cash flow	T_1	T_2
Long forward	0	$-K$	$+1$
Long T_2-bond, short K T_1-bonds	$KP(t,T_1)-P(t,T_2)$	$-K$	$+1$

Setting the portfolio's initial cash flow to zero gives the fair K. If a zero-premium quoted contract instead has $K > P(t,T_2)/P(t,T_1)$, take the bond portfolio and *short* the forward: the future cash flows cancel and $KP(t,T_1) - P(t,T_2) > 0$ is received now. For K below fair, reverse both positions. This idealised arbitrage presumes the quoted forward and bond positions are executable on matching terms.

7.5 Activity 4 — From spot prices to a forward rate

Suppose that $P(t,T_1)$, $P(t,T_2)$, and $\tau_1 = \tau(T_1,T_2)$ are known.

1. Derive the forward discount factor from spot prices.
2. Obtain the simply compounded forward rate $F(t;T_1,T_2)$.
3. Obtain the continuously compounded forward rate $F_c(t;T_1,T_2)$.
4. Explain why the two numerical rates differ although no new price information has been introduced.
5. Explain why neither rate is, by definition, a forecast of the future spot rate prevailing at T_1 .

Generated study answer

Given $P_1 = P(t,T_1)$, $P_2 = P(t,T_2)$ and $\delta = \tau(T_1,T_2)$, replication gives $P(t;T_1,T_2) = P_2/P_1$. Solving $P_2/P_1 = (1+F_s\delta)^{-1}$ yields $F_s = (P_1/P_2 - 1)/\delta$; solving $P_2/P_1 = e^{-F_c\delta}$ yields $F_c = \log(P_1/P_2)/\delta$. The rates differ numerically because their compounding definitions differ, although they quote the same forward price. Neither, by definition, forecasts the spot rate that will prevail at T_1; such an interpretation requires assumptions about dynamics and risk premia.

7.6 Activity 5 — Read the term structures

Discuss and correct each statement where necessary:

1. "The discount curve and the zero-rate curve are two independent market inputs."
2. "If the spot-rate curve is increasing, every forward rate must exceed the spot rate with the same end date."
3. "A forward curve is uniquely defined once the valuation date is fixed."
4. "The short rate and the instantaneous forward rate are unrelated modelling objects."
5. "A discount factor greater than one necessarily creates an arbitrage."

For the third statement, describe two different one-dimensional forward curves that can be extracted from the same two-date forward structure.

Generated study answer

1. The discount and zero curves cannot be selected independently once dates and compounding are fixed; they encode the same positive prices.
2. An upward-sloping spot curve does not license a convention-free comparison with a forward quote. Under a common continuous, additive clock, a strictly increasing spot rate between T_1 and T_2 does imply that the interval forward exceeds the spot rate to T_2; the hypotheses matter.
3. A forward curve is not uniquely a one-argument curve without another rule. One may fix T_1 and vary T_2, or fix a three-month tenor and vary T_1 while generating T_2 contractually. The *instantaneous* forward curve at fixed t is one-dimensional in T.
4. The short rate is the boundary value $f(t,t)$, not an unrelated additional price input when the required limit exists.
5. A discount factor greater than one may reflect a negative implied rate. It is not, by itself, an arbitrage.

Key idea

Exam check: label each quantity as a dated *price*, a *rate quote*, or a *future realised rate*. Many incorrect arguments silently exchange these categories.

7.7 Optional AI-supported discussion

After attempting an activity, ask an AI tutor: "Check whether I have correctly distinguished contracts, prices and rate representations in this problem. Ask me one question at a time if my reasoning is incomplete. Keep the notation of these notes, in particular $\tau_i = \tau(T_{i-1},T_i)$, and do not introduce models or instruments beyond this chapter." Compare its feedback with the timelines, definitions and no-arbitrage relations above; resolve disagreements by returning first to the contractual cash flows.

Part II
Code

8 Notebook 01: dates, conventions and market curves

The supplied notebook is reproduced in execution order. Explanatory markdown and all code cells are retained, including the student exercise placeholders and diagnostic questions. The original notebook and two original LSEG spreadsheets accompany this project. They were not executed as part of the document build; printed notebook outputs are source observations, not a newly validated execution.

Fixed Income — QFI 2026–27

Reference market date: **24 August 2026**

This notebook builds the operational bridge between dates and the fixed-income primitives introduced in the notes. We first construct dates, year fractions and coupon schedules. We then import an LSEG EURIBOR 6M curve and verify the transformations between zero rates and discount factors.

8.1 Learning objectives

At the end of the notebook, you should be able to:

1. distinguish calendar dates, business dates and adjusted dates;
2. compute year fractions under different day-count conventions;
3. construct a coupon schedule and its accrual fractions τ_i;
4. transform discount factors into equivalent zero rates and vice versa;
5. read the conventions attached to a market curve;
6. compute simple forward rates from discount factors and interpret discrepancies with vendor outputs.

8.2 How we will work

Each cycle follows the same sequence:

1. **Guided example:** we interpret and run an existing block of code.
2. **Your turn:** you modify the example or build a small extension.
3. **Discussion checkpoint:** we compare results and identify the relevant financial mechanism.
4. **A possible solution:** discussed in class and provided separately after the attempt.

The objective is not to copy code, but to learn how to **read, modify, validate and interpret** it.

8.3 0. Setup

The notebook uses standard scientific Python packages and QuantLib. If QuantLib is not available in the active Anaconda environment, install it once from **Anaconda Prompt** with:

```
conda install -c conda-forge quantlib-python
```

After installation, restart the Jupyter kernel.

Source code cell 3

```
from pathlib import Path
Path.cwd()
```

Recorded source output

```
WindowsPath('C:/Users/User/OneDriveUniTo/2026-FIXED-INCOME/notebooks/01-dates-and-conventions')
```

Source code cell 4

```
# Import tools for paths, dates, numerical calculations, tables, plots, and financial conventions.
# Set the display format and plotting style, and define the folder containing the raw data.

from pathlib import Path          # File and folder path management
import datetime as dt             # Date and time-interval management in Python

import numpy as np                # Numerical calculations and operations on vectors and arrays
```

```
import pandas as pd               # Management and analysis of tabular data
import matplotlib.pyplot as plt   # Plot creation
import QuantLib as ql             # Financial tools: dates, calendars, conventions, curves, and instruments

pd.set_option("display.float_format", lambda x: f"{x:,.8f}")
plt.style.use("seaborn-v0_8-whitegrid")

DATA_DIR = Path("data/raw")
```

8.4 Cycle 1 — Dates and calendar conventions

8.4.1 Guided example 1A — Python, pandas and QuantLib dates

The same calendar date can be represented by different Python objects. QuantLib dates are particularly useful because calendars, business-day adjustments and financial schedules are built into the library.

Source code cell 6

```
trade_date_py = dt.date(2026, 8, 24)
trade_date_pd = pd.Timestamp("2026-08-24")
trade_date_ql = ql.Date(24, ql.August, 2026)

print("Python date:   ", trade_date_py, type(trade_date_py))
print("pandas date:   ", trade_date_pd, type(trade_date_pd))
print("QuantLib date: ", trade_date_ql, type(trade_date_ql))
```

Recorded source output

```
Python date:    2026-08-24 <class 'datetime.date'>
pandas date:    2026-08-24 00:00:00 <class 'pandas._libs.tslibs.timestamps.Timestamp'>
QuantLib date:  August 24th, 2026 <class 'QuantLib.QuantLib.Date'>
```

Source code cell 7

```
# Print Python dates

print(trade_date_py.strftime("%d/%m/%Y"))  # 'DAY/MONTH/YEAR'
print(trade_date_py.strftime("%Y-%m-%d"))  # 'YEAR-MONTH-DAY'
print(trade_date_py.strftime("%a"))        # 'WEEKDAY'
```

Recorded source output

```
24/08/2026
2026-08-24
Mon
```

Source code cell 8

```
def py_to_ql(date):
    # Convert datetime.date or pandas.Timestamp to QuantLib.Date.
    date = pd.Timestamp(date)
    return ql.Date(date.day, date.month, date.year)


def ql_to_py(date):
    # Convert QuantLib.Date to datetime.date.
    return dt.date(date.year(), date.month(), date.dayOfMonth())

# Check that converting Python → QuantLib → Python preserves the original date
assert ql_to_py(py_to_ql(trade_date_py)) == trade_date_py
```

Source code cell 9

```
ql_to_py(py_to_ql(trade_date_py)) == trade_date_py
```

Recorded source output

```
True
```

8.4.2 Evaluation date and settlement date

QuantLib uses a global **evaluation date**. It should always be set explicitly. The TARGET calendar then advances the trade date by two **business days**, not two calendar days.

Source code cell 11

```
# Set the global evaluation date in QuantLib
ql.Settings.instance().evaluationDate = trade_date_ql

calendar = ql.TARGET()  # Euro-area TARGET business calendar
settlement_days = 2
settlement_date = calendar.advance(trade_date_ql, settlement_days, ql.Days)

print("Trade date:      ", trade_date_ql)
print("Settlement date: ", settlement_date)
```

Recorded source output

```
Trade date:       August 24th, 2026
Settlement date:  August 26th, 2026
```

Source code cell 12

```
# Check if target date
print(calendar.isBusinessDay(trade_date_ql))
print(calendar.isBusinessDay(settlement_date))
```

Recorded source output

```
True
True
```

8.4.3 Guided example 1B — Calendars and business-day conventions

A contractual date may fall on a non-business day. A business-day convention specifies how that date is adjusted.

- **Following:** move to the next business day.
- **Modified Following:** move forward unless this changes the month; in that case move backward.
- **Preceding:** move to the previous business day.
- **Unadjusted:** do not move the date.

Source code cell 14

```
unadjusted_date = ql.Date(31, ql.October, 2026)  # Saturday

adjustments = {
    "Unadjusted": ql.Unadjusted,
    "Following": ql.Following,
    "Modified Following": ql.ModifiedFollowing,
    "Preceding": ql.Preceding,
}

adjustment_table = pd.DataFrame(
    {
        "Convention": name,
        "Unadjusted Date": ql_to_py(unadjusted_date),
        "Unadjusted Weekday": ql_to_py(unadjusted_date).strftime("%a"),
        "Adjusted Date": ql_to_py(calendar.adjust(unadjusted_date, convention)),
        "Adjusted Weekday": ql_to_py(
            calendar.adjust(unadjusted_date, convention)
        ).strftime("%a"),
    }
    for name, convention in adjustments.items()
)


adjustment_table
```

Recorded source output

	Convention	Unadjusted Date	Unadjusted Weekday	Adjusted Date	Adjusted Weekday
0	Unadjusted	2026-10-31	Sat	2026-10-31	Sat
1	Following	2026-10-31	Sat	2026-11-02	Mon
2	Modified Following	2026-10-31	Sat	2026-10-30	Fri
3	Preceding	2026-10-31	Sat	2026-10-30	Fri

8.4.4 End-of-month rule

The end-of-month rule matters when a sequence of dates is generated from a date that is itself the last business day of a month.

Source code cell 16

```
sample_dates = [
    ql.Date(28, ql.August, 2026),
    ql.Date(31, ql.August, 2026),
    ql.Date(30, ql.September, 2026),
]

eom_table = pd.DataFrame({
    "Date": [ql_to_py(d) for d in sample_dates],
    "Is business day": [calendar.isBusinessDay(d) for d in sample_dates],
    "Is end of month": [calendar.isEndOfMonth(d) for d in sample_dates],
    "Calendar end of month": [ql_to_py(calendar.endOfMonth(d)) for d in sample_dates],
})

eom_table
```

Recorded source output

	Date	Is business day	Is end of month	Calendar end of month
0	2026-08-28	True	False	2026-08-31
1	2026-08-31	True	True	2026-08-31
2	2026-09-30	True	True	2026-09-30

8.4.5 Your turn 1 — Business-day conventions

Choose a date other than 31 October 2026 for which `Following` and `Modified Following` produce different adjusted dates.

1. Apply `Following`, `Modified Following`, `Preceding` and `Unadjusted`.
2. Report the weekday of each adjusted date.
3. Be ready to explain why the two following conventions give different results.

Suggested time: 10 minutes.

Source code cell 18

```
# Write your code here
```

Discussion checkpoint

- Which convention crosses the month boundary?
- Which adjustment would you expect before running the code?
- Is the result determined by the calendar, the convention, or both?

8.5 Cycle 2 — Day-count and compounding conventions

8.5.1 Guided example 2A — Day-count conventions and year fractions

The time between two dates is not determined by the dates alone. It also depends on the day-count convention:

$$\tau(T_1, T_2) = \text{yearFraction}(T_1, T_2).$$

We compare four common conventions over the same interval. For `Actual/Actual` we use the **AFB** variant because it reproduces the convention in the LSEG export used later.

Source code cell 21

```
start_date = ql.Date(24, ql.August, 2026)
end_date = ql.Date(28, ql.August, 2028)

day_counts = {
    "Actual/360": ql.Actual360(),
    "Actual/365 Fixed": ql.Actual365Fixed(),
    "Actual/Actual (AFB)": ql.ActualActual(ql.ActualActual.AFB),
    "30/360 (Bond Basis)": ql.Thirty360(ql.Thirty360.BondBasis),
}

day_count_table = pd.DataFrame(
    {
        "Convention": name,
        "Start Date": ql_to_py(start_date),
        "End Date": ql_to_py(end_date),
```

```
        "Calendar Days": end_date - start_date,
        "Year Fraction": dc.yearFraction(start_date, end_date),
    }
    for name, dc in day_counts.items()
)

day_count_table
```

Recorded source output

	Convention	Start Date	End Date	Calendar Days	Year Fraction
0	Actual/360	2026-08-24	2028-08-28	735	2.04166667
1	Actual/365 Fixed	2026-08-24	2028-08-28	735	2.01369863
2	Actual/Actual (AFB)	2026-08-24	2028-08-28	735	2.01095890
3	30/360 (Bond Basis)	2026-08-24	2028-08-28	735	2.01111111

8.5.2 Your turn 2 — Day-count conventions

Choose a start date and an end date such that the interval includes 29 February. Compute the year fraction under:

- `Actual/360`;
- `Actual/365 Fixed`;
- `Actual/Actual (AFB)`;
- `30/360 (Bond Basis)`.

Present the results in a `DataFrame` and briefly explain why they differ.

Suggested time: 12 minutes.

Source code cell 23

```
# Write your code here
```

Discussion checkpoint

- Which convention produces the largest year fraction?
- Where does the leap day enter the calculation?
- Why can a small difference in τ matter in pricing?

The year fraction is part of the definition of a rate. A quoted rate without its dates and conventions is incomplete.

For a sequence of coupon dates $T_0, T_1, \ldots, T_n$, we use the notation

$$\tau_i = \tau(T_{i-1}, T_i).$$

8.5.3 Guided example 2B — Compounding conventions

Let $P(t,T)$ be the time-t price of a zero-coupon bond paying one unit at T, and let $\tau = \tau(t,T)$.

Simple compounding

$$P(t,T) = \frac{1}{1 + R_s(t,T)\tau}.$$

Periodic compounding with frequency m

$$P(t,T) = \left(1 + \frac{R_m(t,T)}{m}\right)^{-m\tau}.$$

Continuous compounding

$$P(t,T) = e^{-R_c(t,T)\tau}.$$

The rates are numerically different but financially equivalent when they produce the same discount factor.

Source code cell 27

```
def discount_from_rate(rate, tau, compounding="continuous", frequency=1):
    if compounding == "simple":
        return 1.0 / (1.0 + rate * tau)
    if compounding == "periodic":
        return (1.0 + rate / frequency) ** (-frequency * tau)
    if compounding == "continuous":
        return np.exp(-rate * tau)
    raise ValueError("Unknown compounding convention")


def rate_from_discount(discount_factor, tau, compounding="continuous", frequency=1):
    if tau <= 0:
        return np.nan
    if compounding == "simple":
        return (1.0 / discount_factor - 1.0) / tau
    if compounding == "periodic":
        return frequency * (discount_factor ** (-1.0 / (frequency * tau)) - 1.0)
    if compounding == "continuous":
        return -np.log(discount_factor) / tau
    raise ValueError("Unknown compounding convention")
```

Source code cell 28

```
tau_example = 5.0
discount_example = 0.85

equivalent_rates = pd.DataFrame({
    "Convention": ["Simple", "Annual", "Semiannual", "Continuous"],
    "Rate": [
        rate_from_discount(discount_example, tau_example, "simple"),
        rate_from_discount(discount_example, tau_example, "periodic", 1),
        rate_from_discount(discount_example, tau_example, "periodic", 2),
        rate_from_discount(discount_example, tau_example, "continuous"),
    ],
})
equivalent_rates["Rate Percent"] = 100 * equivalent_rates["Rate"]
equivalent_rates
```

Recorded source output

	Convention	Rate	Rate Percent
0	Simple	0.03529412	3.52941176
1	Annual	0.03303780	3.30378041
2	Semiannual	0.03276935	3.27693466
3	Continuous	0.03250379	3.25037859

8.5.4 Your turn 3 — Equivalent interest rates

For a five-year discount factor $P(t, T) = 0.85$, calculate the equivalent:

- simply compounded zero rate;
- annually compounded zero rate;
- semiannually compounded zero rate;
- continuously compounded zero rate.

Verify numerically that all four rates reproduce the same discount factor.

Suggested time: 12 minutes.

Source code cell 30

```
# Write your code here
```

Discussion checkpoint

- Why are the four rates different?
- Which quantity remains unchanged?
- Which rate is the largest in this example, and why?

8.6 Cycle 3 — Coupon schedules

8.6.1 Guided example 3 — A coupon schedule and the accrual fractions τ_i

We now generate a semiannual schedule. The code remains visible rather than being hidden in an external function: the purpose is to understand exactly which conventions enter the construction.

Before constructing the schedule, keep four distinct dates conceptually separate:

Date	Meaning
`evaluationDate`	The date at which the instrument is valued
Trade date	The date on which the transaction is agreed
Settlement date	The date on which the transaction settles
Schedule `effectiveDate`	The date from which the schedule starts

The schedule `effectiveDate` is therefore best understood as the **schedule start date**. It need not coincide with the evaluation, trade or settlement date, and it may lie in the past. For an existing instrument, we start the schedule from the previous coupon date so that the current coupon period is represented correctly.

For standard plain-vanilla bonds and swaps, generating the schedule backward from the contractual maturity date is the natural convention and is the rule used here.

Source code cell 33

```
previous_coupon_date = ql.Date(15, ql.June, 2026)
maturity_date = ql.Date(15, ql.December, 2029)
tenor = ql.Period(6, ql.Months)
day_rolling = ql.ModifiedFollowing
termination_day_rolling = ql.ModifiedFollowing
date_generation = ql.DateGeneration.Backward
end_of_month = False

schedule = ql.Schedule(
    previous_coupon_date,  # Start of the current coupon period
    maturity_date,
    tenor,
    calendar,
    day_rolling,
    termination_day_rolling,
    date_generation,
    end_of_month,
)

schedule_dates = list(schedule)
accrual_day_count = ql.Actual360()

schedule_table = pd.DataFrame({
    "Period": np.arange(1, len(schedule_dates)),
    "Coupon Period Start": [ql_to_py(d) for d in schedule_dates[:-1]],
    "Coupon Period End": [ql_to_py(d) for d in schedule_dates[1:]],
    "Tau_i": [
        accrual_day_count.yearFraction(start, end)
        for start, end in zip(schedule_dates[:-1], schedule_dates[1:])
    ],
})

schedule_table
```

Recorded source output

	Period	Coupon Period Start	Coupon Period End	Tau_i
0	1	2026-06-15	2026-12-15	0.50833333
1	2	2026-12-15	2027-06-15	0.50555556
2	3	2027-06-15	2027-12-15	0.50833333
3	4	2027-12-15	2028-06-15	0.50833333
4	5	2028-06-15	2028-12-15	0.50833333
5	6	2028-12-15	2029-06-15	0.50555556
6	7	2029-06-15	2029-12-17	0.51388889

8.6.2 Your turn 4 — Coupon schedules

Change **one** input in the schedule: maturity date, end-of-month rule or business-day convention. Keep backward generation. Then:

1. generate the new schedule;

2. calculate the accrual fractions τ_i;
3. identify any stub period;
4. compare the result with the original schedule.

Different groups may choose different inputs.

Suggested time: 15 minutes.

Source code cell 35

```
# Write your code here
```

8.6.3 Discussion checkpoint

1. Is the first coupon period regular or a stub?
2. Are all the accrual fractions τ_i equal?
3. Why does the schedule begin before the settlement date?
4. What changes when the maturity date falls on a weekend?

8.7 Cycle 4 — Reading and validating a market curve

8.7.1 Guided example 4A — The LSEG EURIBOR 6M curve

We now have the tools required to interpret a market curve.

The supplied LSEG curve has the following settings:

- curve: **EUR — Deposits and IRS versus EURIBOR 6M**;
- trade date: **24 August 2026**;
- collateralized: **Off**;
- discounting tenor: **6M**;
- interpolation: **Cubic Discount**;
- extrapolation: **None**;
- price side: **Mid**;
- zero-rate calculation: **Actual/Actual, compounded**;
- forward-rate style: **FRA Actual/360**.

The curve is useful as a pedagogical single-curve representation. It should not be confused with the €STR discounting curve used for collateralized EUR derivatives in a modern multi-curve framework.

Source code cell 38

```
ZERO_CURVE_FILE = DATA_DIR / "lseg_euribor_6m_zero_curve_2026-08-24.xlsx"
FORWARD_CURVE_FILE = DATA_DIR / "lseg_euribor_6m_forward_curve_2026-08-24.xlsx"

raw_curve = pd.read_excel(
    ZERO_CURVE_FILE,
    sheet_name="Table",
    header=1,
    usecols="D:I",
)

raw_curve.head()
```

Recorded source output

	Tenor	End Date	Rate Percent	Discount Factor	Instrument Code 1	Mid 1
0	0D	2026-08-24	2.30739657	1.00000000	NaN	NaN
1	1D	2026-08-25	2.30739657	0.99993750	EUROND=	2.25000000
2	1W	2026-09-02	2.24630482	0.99945240	EURIBORSWD=	NaN
3	1M	2026-09-28	2.27658346	0.99784378	EURIBOR1MD=	NaN
4	3M	2026-11-26	2.59043047	0.99343534	EURIBOR3MD=	NaN

Source code cell 39

```
curve = raw_curve.rename(columns={
    "Tenor": "Tenor",
    "End Date": "Maturity Date",
    "Rate Percent": "Zero Rate Percent",
    "Discount Factor": "LSEG Discount Factor",
    "Instrument Code 1": "Instrument Code",
    "Mid 1": "Instrument Mid",
```

```
}).copy()

curve = curve.dropna(subset=["Maturity Date", "LSEG Discount Factor"])
curve["Maturity Date"] = pd.to_datetime(curve["Maturity Date"])
curve["Zero Rate"] = curve["Zero Rate Percent"] / 100.0
curve = curve.reset_index(drop=True)

curve.head(10)
```

Recorded source output

	Tenor	Maturity Date	Zero Rate Percent	LSEG Discount Factor	Instrument Code	Instrument Mid	Zero Rate
0	0D	2026-08-24	2.30739657	1.00000000	NaN	NaN	0.02307397
1	1D	2026-08-25	2.30739657	0.99993750	EUROND=	2.25000000	0.02307397
2	1W	2026-09-02	2.24630482	0.99945240	EURIBORSWD=	NaN	0.02246305
3	1M	2026-09-28	2.27658346	0.99784378	EURIBOR1MD=	NaN	0.02276583
4	3M	2026-11-26	2.59043047	0.99343534	EURIBOR3MD=	NaN	0.02590430
5	4M	2026-12-16	2.65325468	0.99185451	NaN	NaN	0.02653255
6	cm2	2027-03-16	2.75197777	0.98494145	FEIcm2	NaN	0.02751978
7	cm3	2027-06-17	2.83459838	0.97751246	FEIcm3	NaN	0.02834598
8	cm4	2027-09-16	2.89317644	0.97013663	FEIcm4	NaN	0.02893176
9	cm5	2027-12-15	2.93012344	0.96288515	FEIcm5	NaN	0.02930123

8.7.2 Guided example 4B — From zero rates to discount factors

The LSEG numbers identify the precise convention empirically. They are reproduced by:

- `Actual/Actual (AFB)` for $\tau(t,T)$;
- annual periodic compounding ($m = 1$).

Therefore,

$$P(t,T) = (1 + R(t,T))^{-\tau(t,T)}.$$

Notice that the curve label "6M" identifies the EURIBOR tenor; it does **not** imply semiannual compounding of the exported zero rate.

Source code cell 41

```
lseg_zero_day_count = ql.ActualActual(ql.ActualActual.AFB)

curve["Tau Act/Act AFB"] = curve["Maturity Date"].apply(
    lambda maturity: lseg_zero_day_count.yearFraction(
        trade_date_ql, py_to_ql(maturity)
    )
)

curve["Python Discount Factor"] = np.where(
    curve["Tau Act/Act AFB"] == 0,
    1.0,
    discount_from_rate(
        curve["Zero Rate"],
        curve["Tau Act/Act AFB"],
        compounding="periodic",
        frequency=1,
    ),
)

curve["DF Error"] = (
    curve["Python Discount Factor"] - curve["LSEG Discount Factor"]
)

curve[[
    "Tenor", "Maturity Date", "Tau Act/Act AFB", "Zero Rate Percent",
    "LSEG Discount Factor", "Python Discount Factor", "DF Error"
]].head(12)
```

Recorded source output

	Tenor	Maturity Date	Tau Act/Act AFB	Zero Rate Percent	LSEG Discount Factor	Python Discount Factor	DF Error
0	0D	2026-08-24	0.00000000	2.30739657	1.00000000	1.00000000	0.00000000
1	1D	2026-08-25	0.00273973	2.30739657	0.99993750	0.99993750	-0.00000000
2	1W	2026-09-02	0.02465753	2.24630482	0.99945240	0.99945240	-0.00000000
3	1M	2026-09-28	0.09589041	2.27658346	0.99784378	0.99784378	0.00000000
4	3M	2026-11-26	0.25753425	2.59043047	0.99343534	0.99343534	0.00000000
5	4M	2026-12-16	0.31232877	2.65325468	0.99185451	0.99185451	-0.00000000

6	cm2	2027-03-16	0.55890411	2.75197777	0.98494145	0.98494145	-0.00000000
7	cm3	2027-06-17	0.81369863	2.83459838	0.97751246	0.97751246	0.00000000
8	cm4	2027-09-16	1.06301370	2.89317644	0.97013663	0.97013663	-0.00000000
9	cm5	2027-12-15	1.30958904	2.93012344	0.96288515	0.96288515	-0.00000000
10	cm6	2028-03-15	1.55616438	2.95673173	0.95566819	0.95566819	-0.00000000
11	2Y	2028-08-28	2.01095890	3.07776329	0.94086162	0.94086162	-0.00000000

Source code cell 42

```
print(f"Maximum absolute DF error: {curve['DF Error'].abs().max():.3e}")

assert curve["DF Error"].abs().max() < 1e-12
```

Recorded source output

```
Maximum absolute DF error: 9.437e-16
```

8.7.3 Your turn 5 — Validate selected LSEG discount factors

Select the maturities 1Y, 5Y, 10Y and 30Y. For each maturity:

1. report $\tau(t,T)$ under `Actual/Actual (AFB)`;
2. reconstruct the discount factor from the LSEG zero rate;
3. compute the absolute pricing error;
4. verify that the error is negligible.

Suggested time: 15 minutes.

Source code cell 44

```
# Write your code here
```

Discussion checkpoint

- What does a negligible error validate?
- Would the same result hold under `Actual/360` or semiannual compounding?
- Why is an automated assertion useful here?

The assertion is a reproducibility check. If it fails after changing the dataset or conventions, the relationship between rates and discount factors must be investigated before proceeding.

8.7.4 Guided example 4C — Equivalent representations of the term structure

The discount factor is the primitive pricing object. Different rate conventions are alternative representations of the same collection of zero-coupon bond prices.

Source code cell 48

```
positive_tau = curve["Tau Act/Act AFB"] > 0

curve["Simple Zero Rate"] = np.nan
curve["Continuous Zero Rate"] = np.nan

curve.loc[positive_tau, "Simple Zero Rate"] = (
    1.0 / curve.loc[positive_tau, "LSEG Discount Factor"] - 1.0
) / curve.loc[positive_tau, "Tau Act/Act AFB"]

curve.loc[positive_tau, "Continuous Zero Rate"] = (
    -np.log(curve.loc[positive_tau, "LSEG Discount Factor"])
    / curve.loc[positive_tau, "Tau Act/Act AFB"]
)

selected_tenors = ["1Y", "2Y", "5Y", "10Y", "20Y", "30Y"]
comparison = curve.loc[curve["Tenor"].isin(selected_tenors), [
    "Tenor", "Maturity Date", "LSEG Discount Factor", "Zero Rate",
    "Simple Zero Rate", "Continuous Zero Rate"
]].copy()

comparison
```

Recorded source output

	Tenor	Maturity Date	LSEG Discount Factor	Zero Rate	Simple Zero Rate	Continuous Zero Rate
11	2Y	2028-08-28	0.94086162	0.03077763	0.03125651	0.03031350
14	5Y	2031-08-26	0.85724236	0.03125159	0.03326979	0.03077320
19	10Y	2036-08-26	0.72340916	0.03288959	0.03821342	0.03236030
25	20Y	2046-08-27	0.50463819	0.03477254	0.04906073	0.03418163
27	30Y	2056-08-28	0.37312809	0.03339461	0.05598104	0.03284912

Source code cell 49

```
fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))

axes[0].plot(curve["Maturity Date"], 100 * curve["Zero Rate"], marker="o")
axes[0].set_title("LSEG zero curve")
axes[0].set_ylabel("Zero rate (%)")
axes[0].set_xlabel("Maturity date")

axes[1].plot(curve["Maturity Date"], curve["LSEG Discount Factor"], marker="o")
axes[1].set_title("LSEG discount factors")
axes[1].set_ylabel("Discount factor")
axes[1].set_xlabel("Maturity date")

fig.autofmt_xdate()
fig.tight_layout()
plt.show()
```

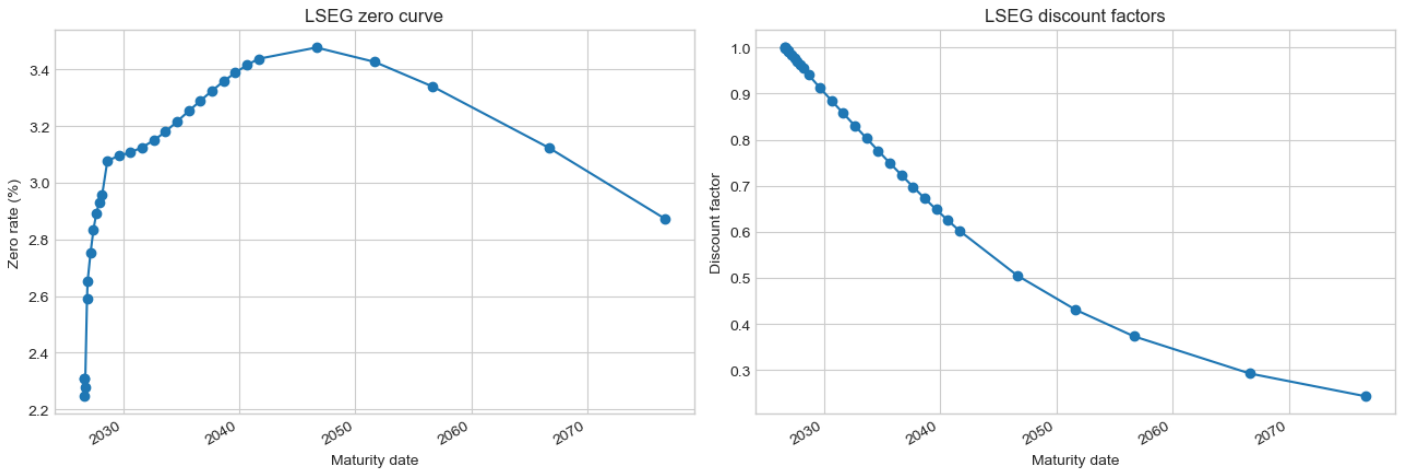


Figure 1: Source notebook plot: zero-rate and discount-factor term structures from the supplied 24 August 2026 LSEG data. Different vertical scales describe the same curve.

Recorded source output

8.7.5 Your turn 6 — Interpret the term structure

Using the LSEG data:

1. identify the maturity at which the zero rate reaches its maximum;
2. describe the short, medium and long ends of the curve;
3. explain why discount factors generally decrease with maturity;
4. discuss the unusual behaviour beyond 30 years.

Write four to six lines of economic interpretation. Do not merely describe the mechanics of the graph.

Suggested time: 10 minutes, or complete after class.

Your interpretation:

Write here.

8.7.6 Guided example 4D — Forward rates from discount factors

For $T_1 < T_2$, the simply compounded forward rate is

$$F(t; T_1, T_2) = \frac{1}{\tau(T_1, T_2)} \left[\frac{P(t, T_1)}{P(t, T_2)} - 1 \right].$$

For consecutive dates T_{i-1} and T_i, we write

$$\tau_i = \tau(T_{i-1}, T_i).$$

The following calculation uses consecutive exported curve nodes and `Actual/360`. These are **node-to-node forwards**, not necessarily standard 6M FRA rates.

Source code cell 53

```
forward_day_count = ql.Actual360()

node_forwards = curve.loc[positive_tau, [
    "Tenor", "Maturity Date", "LSEG Discount Factor"
]].copy().reset_index(drop=True)

node_forwards["Previous Date"] = node_forwards["Maturity Date"].shift(1)
node_forwards["Previous Discount Factor"] = node_forwards["LSEG Discount Factor"].shift(1)

node_forwards["Tau_i"] = node_forwards.apply(
    lambda row: np.nan if pd.isna(row["Previous Date"]) else
    forward_day_count.yearFraction(
        py_to_ql(row["Previous Date"]), py_to_ql(row["Maturity Date"])
    ),
    axis=1,
)

node_forwards["Forward Rate"] = (
    node_forwards["Previous Discount Factor"]
    / node_forwards["LSEG Discount Factor"] - 1.0
) / node_forwards["Tau_i"]

node_forwards.head(12)
```

Recorded source output

	Tenor	Maturity Date	LSEG Discount Factor	Previous Date	Previous Discount Factor	Tau_i	Forward Rate
0	1D	2026-08-25	0.99993750	NaT	NaN	NaN	NaN
1	1W	2026-09-02	0.99945240	2026-08-25	0.99993750	0.02222222	0.02184182
2	1M	2026-09-28	0.99784378	2026-09-02	0.99945240	0.07222222	0.02232126
3	3M	2026-11-26	0.99343534	2026-09-28	0.99784378	0.16388889	0.02707671
4	4M	2026-12-16	0.99185451	2026-11-26	0.99343534	0.05555556	0.02868866
5	cm2	2027-03-16	0.98494145	2026-12-16	0.99185451	0.25000000	0.02807500
6	cm3	2027-06-17	0.97751246	2027-03-16	0.98494145	0.25833333	0.02941892
7	cm4	2027-09-16	0.97013663	2027-06-17	0.97751246	0.25277778	0.03007735
8	cm5	2027-12-15	0.96288515	2027-09-16	0.97013663	0.25000000	0.03012396
9	cm6	2028-03-15	0.95566819	2027-12-15	0.96288515	0.25277778	0.02987500
10	2Y	2028-08-28	0.94086162	2028-03-15	0.95566819	0.46111111	0.03412897
11	3Y	2029-08-27	0.91238480	2028-08-28	0.94086162	1.01111111	0.03086843

8.7.7 LSEG forward-curve analysis

LSEG also exports standard 6M FRA rates. We read them separately because their start and end dates generally do not coincide with all the sampled zero-curve nodes.

Source code cell 55

```
raw_forward = pd.read_excel(FORWARD_CURVE_FILE, sheet_name="Forward Curve Analysis", header=None)

# Columns in the LSEG export:
# D = start interval, E = start date, J = 6M end date, K = 6M forward rate.
lseg_forward_6m = raw_forward.iloc[6:, [3, 4, 9, 10]].copy()
lseg_forward_6m.columns = ["Start Interval", "Start Date", "End Date", "LSEG Forward Percent"]
lseg_forward_6m = lseg_forward_6m.dropna(subset=["Start Date", "End Date", "LSEG Forward Percent"])
lseg_forward_6m["Start Date"] = pd.to_datetime(lseg_forward_6m["Start Date"])
lseg_forward_6m["End Date"] = pd.to_datetime(lseg_forward_6m["End Date"])
lseg_forward_6m["Tau Act/360"] = lseg_forward_6m.apply(
    lambda row: forward_day_count.yearFraction(
        py_to_ql(row["Start Date"]), py_to_ql(row["End Date"])
    ),
    axis=1,
)

lseg_forward_6m.head(10)
```

Recorded source output

	Start Interval	Start Date	End Date	LSEG Forward Percent	Tau Act/360
6	2026-08-24 00:00:00	2026-08-26	2027-02-26	2.64454721	0.51111111
7	1W	2026-09-02	2027-03-02	2.66695967	0.50277778
8	1M	2026-09-28	2027-03-30	2.76121469	0.50833333
9	2M	2026-10-26	2027-04-26	2.79915159	0.50555556
10	3M	2026-11-26	2027-05-26	2.83260672	0.50277778
11	4M	2026-12-28	2027-06-28	2.90023330	0.50555556
12	5M	2027-01-26	2027-07-26	2.93518477	0.50277778
13	6M	2027-02-26	2027-08-31	2.96520508	0.51666667
14	9M	2027-05-26	2027-11-26	3.01038844	0.51111111
15	1Y	2027-08-26	2028-02-28	2.99410998	0.51666667

8.7.8 Diagnostic observation

The standard LSEG 6M forward rates cannot in general be reconstructed from the **sparse set of exported discount-factor nodes alone** without reproducing the vendor's complete interpolation and curve machinery. Moreover, the forward-analysis object may embody tenor-specific conventions not visible in the sampled zero table.

This is not a numerical failure to conceal. It is an important model-validation lesson:

> before comparing two calculated rates, verify that dates, day counts, compounding, interpolation and the underlying curve object are identical.

In a later notebook we will construct an explicit curve object and perform a controlled comparison in which every convention is specified by us.

8.8 10. Conclusions

1. Dates and conventions are part of the definition of every interest rate.
2. The same discount factor can be represented by simple, periodic or continuously compounded zero rates.
3. The supplied LSEG discount factors are reproduced by `Actual/Actual (AFB)` and annual compounding.
4. The label "EURIBOR 6M" concerns the index tenor, not the compounding frequency of the zero rate.
5. Forward rates require discount factors at both endpoints and an accrual fraction τ_i.
6. Vendor outputs should be validated, not merely copied: interpolation and hidden curve conventions matter.

8.8.1 Suggested experiments

- Change the day-count convention and measure the effect on the reconstructed discount factors.
- Replace annual compounding with semiannual compounding.
- Change the maturity date of the schedule and inspect the stub.
- Restrict the plot to maturities up to 30 years and discuss the long end separately.

8.9 Homework — A convention audit

Prepare a short, reproducible notebook that documents the convention choices for the following fixed-income problem.

8.9.1 Part A — Settlement and adjusted dates

Use 24 August 2026 as the trade date and the TARGET calendar.

1. Compute the two-business-day settlement date.
2. Choose a contractual date that falls on a non-business day.
3. Adjust it under `Following`, `Modified Following` and `Preceding`.
4. Report the weekday before and after each adjustment.

8.9.2 Part B — Coupon schedule

Construct a semiannual schedule from the settlement date to 31 August 2031 using:

- TARGET calendar;
- `Modified Following`;

- `Actual/360` accrual fractions;
- backward date generation;
- no end-of-month rule.

Report the dates and the accrual fractions τ_i. Identify any irregular period and explain why it appears. Then generate the same schedule using forward generation and compare the results.

8.9.3 Part C — Equivalent rates

Extract the 5Y LSEG discount factor. Calculate the equivalent:

- simply compounded zero rate;
- annually compounded zero rate;
- semiannually compounded zero rate;
- continuously compounded zero rate.

Reconstruct the discount factor from each rate and report the absolute errors.

8.9.4 Part D — Validation note

Write a short paragraph (approximately 100 words) explaining why a rate cannot be interpreted or validated without its dates, day-count convention and compounding convention.

8.9.5 Deliverable

Submit one executed notebook containing:

- clearly labelled input assumptions;
- the required tables;
- reproducibility checks using `assert` where appropriate;
- concise comments on the financial meaning of the results.

Expected working time: 45–60 minutes.

9 Python lab: conventions, source-data checks and supporting guides

9.1 Classroom map

Follow the chain contractual dates, adjusted dates, year fraction, compounding and discount factor. Keep the evaluation date (valuation), trade date (agreement), settlement date (exchange after the contractual business-day lag) and maturity date (termination or final payment) distinct. The source's examples often use two business days for settlement; this is not a universal lag for every instrument. For each topic: explain and annotate, run and read, try the exercise, discuss and validate. The indicative classroom plan allocates 35–45 minutes to opening/questionnaire/materials, 40–45 to dates/TARGET/day rolling, 35–40 to day count and exercises, 10 to a break, 35–45 to compounding and exercises, remaining time to coupon schedules, and 5 to discussion. Coupon schedules are useful but optional for that session; LSEG analysis follows the introduction of forward rates.

9.2 Reading the original LSEG data correctly

The supplied curve is EUR deposits and IRS versus EURIBOR 6M, valued on 24 August 2026. The notebook records collateral Off, discounting tenor 6M, CubicDiscount interpolation, no extrapolation and Mid prices. Zero rates use Actual/Actual (AFB) and annual compounding; FRA forward rates use Actual/360. The 6M index tenor does not mean semiannual compounding of the exported zero rate. The original notebook and workbooks are included. Run the notebook from the project root with its original files under `data/raw`; Python requires NumPy, Pandas, Matplotlib, QuantLib and an Excel reader such as openpyxl. The printed outputs in the notebook chapter are supplied outputs. They are not evidence of a newly executed QuantLib notebook.

Worked example

An independent check of the original workbook's 5Y, 10Y and 30Y nodes reconstructs $P = (1+y)^{-\tau}$ after dividing the percentage quote by 100. The rates and discount factors are:

Tenor	Zero rate (%)	τ AFB	Discount factor
5Y	3.125158602	$5 + 2/365$	0.8572423570
10Y	3.288958602	$10 + 2/365$	0.7234091624
30Y	3.339460615	$30 + 4/365$	0.3731280911

The maturity dates are 26 August 2031, 26 August 2036 and 28 August 2056 respectively, rather than anniversaries of 24 August. Calendar dates therefore matter even for a label such as 5Y.

A source-data mismatch to resolve in Exercise 5. The sheet requests a 1Y node, but the supplied export contains no exact 1Y/12M row. Its short nodes include money-market futures labels `cm2`, `cm3`, etc., followed later by standard year tenors. Do not silently substitute a futures row for 1Y. Obtain the matching export or state and implement an interpolation rule; the latter is a new calculation, not a supplied 1Y observation. The exported zero rate reaches its maximum at 20Y (approximately 3.477254%) and then declines: about 3.339461% at 30Y, 3.123653% at 40Y and 2.872986% at 50Y. The associated discount factors still decrease. A falling annualised rate at a longer maturity does not by itself imply an increasing discount factor: the accrual horizon is also longer. The supplied notebook plot is retained beside its code. Node-to-node forward rates reconstructed from consecutive exported dates use those exact intervals. They are not automatically the vendor's constant-6M forward curve. Replicating that curve also requires interpolation, schedules and market conventions.

9.3 The six supplied student exercises

Exercise 1: business-day conventions. Choose an unadjusted date for which Following and Modified Following differ. Apply Following, Modified Following, Preceding and Unadjusted. Report each weekday and explain why the first two results differ.

Exercise 2: day counts. Choose start and end dates including 29 February. Calculate Actual/360, Actual/365 Fixed, Actual/Actual AFB and 30/360 Bond Basis year fractions. Present a DataFrame and explain the differences.

Exercise 3: coupon schedules. Starting from the notebook schedule, change one input at a time: maturity, end-of-month rule, business-day convention. Retain backward generation. Generate each new schedule, calculate τ_i, identify any stub and explain the differences from the original.

Exercise 4: equivalent rates. For a five-year discount factor 0.85, calculate simple, annual, semiannual and continuous zero rates. Verify that all reproduce the discount factor and explain why their quoted numbers differ.

Worked example

Generated check. With $\tau = 5$, $L = (1/0.85 - 1)/5 = 3.529412\%$, $Y = 0.85^{-1/5} - 1 = 3.303780\%$, $Y_2 = 2(0.85^{-1/10} - 1) = 3.276935\%$ and $R = -\log(0.85)/5 = 3.250379\%$. Substitute each into its own discounting formula; mixing a rate with another compounding convention will fail.

Exercise 5: LSEG validation. Select 1Y, 5Y, 10Y and 30Y. Calculate AFB τ, convert percentages to decimals, reconstruct annual-compounding discount factors, compare with the export and report absolute errors in a table. Address the missing exact 1Y row as explained above.

Exercise 6: interpret the term structure. Plot zero rates and discount factors, locate the maximum zero rate, describe short/medium/long ends, explain why discount factors generally decrease, and discuss the unusual shape beyond 30Y. Provide both graphs and an economic interpretation.

9.4 LSEG export workflow supplied for classroom verification

The source labels this a *provisional* workflow: command names and positions must be verified in the installed Workspace version.

1. Open Workspace and search for Curve Builder.
2. Select EUR, EURIBOR, 6M tenor, deposits and IRS versus EURIBOR 6M.
3. Set/verify 24 August 2026 to reproduce this dataset.

4. Inspect short, intermediate and long constituents, including deposits, EURIBOR futures and swaps.
5. Record price side, discounting tenor/collateral, interpolation, extrapolation, zero-rate day count and compounding.
6. Locate the zero-curve results; inspect tenor, maturity date, zero rate, discount factor and instrument identifier where available.
7. Use the available export/download/Excel command; verify its exact label.
8. Save with a transparent name, e.g. `lseg_euribor_6m_zero_curve_2026-08-24.xlsx`.
9. Retain the unmodified export and worksheet names under the lab's `data/raw` directory (the source course path is `01-dates-and-conventions/data/raw`).

The six supplied verification items are: exact application/view name, exact curve description, valuation-date control, results-table location/name, export command/workbook structure, and conventions/settings. The source deliberately postpones forward-curve analysis and export until forward rates have been introduced.

9.5 NumPy array syntax

The expressions below are reference snippets rather than one executable program. Choose compatible array shapes; individual examples need their own inputs.

```
Create Arrays
a = np.array([1, 2, 3]) -- 1D array
b = np.array([[1, 2], [3, 4]]) -- 2D array (matrix)
Special Arrays
np.zeros(5) → [0. 0. 0. 0. 0.]
np.ones((2,3)) → 2x3 matrix of ones
np.eye(3) → identity matrix
np.arange(0, 10, 2) → [0 2 4 6 8]
np.linspace(0, 1, 5) → [0. 0.25 0.5 0.75 1.]
Shape & Type
a.shape -- (n,) or (m,n)
a.ndim -- number of dimensions
a.size -- total number of elements
a.dtype -- data type (int, float, etc.)
Indexing & Slicing
a[0] -- first element
a[-1] -- last element
a[1:4] -- slice
a[:3] -- first three
a[::2] -- every 2nd element
2D Indexing
b[0,1] -- element (0,1)
b[:,1] -- 2nd column
b[1,:] -- 2nd row
b[0:2,0:2] -- top-left 2x2 block
Operations
a + b -- elementwise addition
a * b -- elementwise multiplication
a @ b -- matrix product
a.T -- transpose
np.dot(a,b) -- dot product
Functions
np.sqrt(a), np.exp(a), np.log(a)
np.sum(a), np.mean(a), np.max(a)
Reshape & Combine
a.reshape(3,2)
np.hstack([a,b]) -- horizontal
np.vstack([a,b]) -- vertical
np.concatenate([a,b],axis=0)
Boolean Masks
mask = a > 2
a[mask] → filtered elements
a[a % 2 == 0] → even numbers
```

9.6 Pandas syntax

These source examples illustrate separate operations. Define the DataFrames used in each fragment and run only the operations appropriate to the current columns.

```
Import
 import pandas as pd
Create a DataFrame
 data = {"Time": [0.5, 1.0, 1.5, 2.0],
         "Rate": [0.02, 0.025, 0.027, 0.03]}
 df = pd.DataFrame(data)
Read / Write
 df = pd.read_excel("file.xlsx", sheet_name="Sheet1")
 df = pd.read_csv("file.csv")
 df.to_excel("output.xlsx", index=False)
 df.to_csv("output.csv", index=False)  # no index column in file
Select Data
 # Columns
 df["Rate"]            # Series
 df[["Time","Rate"]]   # DataFrame

 # Rows by label/position
```

```
 df.loc[1]             # label
 df.iloc[0]            # position

 # Rows + columns
 df.loc[0:2, ["Time","Rate"]]
 df.iloc[0:2, 0:2]
Add / Modify / Drop
 df["Discount"] = 1 / (1 + df["Rate"] * df["Time"])  # add/modify
 df = df.drop(columns=["Rate"])                      # drop column
 df = df.drop(index=[0])                             # drop row
 df = df.rename(columns={"Time":"Maturity"})         # rename
Vector Ops & Filters
 df["Forward"] = df["Rate"] + 0.001    # elementwise
 df_sorted = df.sort_values("Rate", ascending=False)
 df_filt = df[df["Rate"] > 0.025]
Concat & Merge
 new_row = pd.DataFrame([[2.5, 0.031]], columns=["Time","Rate"])
 df2 = pd.concat([df, new_row], ignore_index=True)
 merged = pd.merge(df1, df2, on="Time", how="inner")  # joins: inner/left/right/outer
Quick Inspect
 df.info()       # structure & dtypes
 df.describe()   # stats
 df.head(5)      # first rows
 df.tail(3)      # last rows
Indexing Tips
 # loc uses labels; iloc uses integer positions
 # Use df.reset_index(drop=True) to reset to 0..n-1
 # Set index: df.set_index("Time", inplace=True)

I/O Extras
 # CSV options
 pd.read_csv("f.csv", usecols=["Time","Rate"])
 # Excel writer overwrite (single sheet)
 with pd.ExcelWriter("book.xlsx", engine="openpyxl", mode="a", if_sheet_exists="replace") as w:
     df.to_excel(w, sheet_name="Sheet1", index=False)
```

9.7 Jupyter: two modes and recovery

Escape enters Command Mode (manage cells); Enter enters Edit Mode (write text). In Command Mode, use the letter alone, not Ctrl plus the letter.

Keys	Action
Shift+Enter / Ctrl+Enter	Run and move to next cell / run and remain.
Alt+Enter / Ctrl+S	Run and insert below / save.
A / B	Insert above / below (Command Mode).
D,D / Z	Delete selected cells / undo cell deletion.
X / C / V / Shift+V	Cut / copy / paste below / paste above.
M / Y / R	Change cell to Markdown / Code / Raw.
Up/Down; Shift+Up/Down	Move; extend selection.
Shift+M / L	Merge selected cells / toggle line numbers.
Tab / Shift+Tab	Indent or completion / object or function information (Edit Mode).
Ctrl+/; Ctrl+] / Ctrl+[	Toggle comment; indent / dedent lines.
Ctrl+Z / Ctrl+Y / Ctrl+A	Undo / redo text edits / select cell text.
I,I / 0,0 / H	Interrupt / restart kernel / shortcuts help (Command Mode).

If a cell is stuck, Escape then I,I; if needed restart. Restarting clears variables, so rerun required cells from the top. Save before restarting or closing. Shortcuts can vary across Notebook/JupyterLab versions; consult Help or H for the installed version.

A Course roadmap

The supplied syllabus organises the course around the following topics:

- spot and forward contracts, including the taxonomy of rates;
- benchmark market rates: the LIBOR environment before and after the 2007 credit crisis and the reform from LIBOR to risk-free rates (RFRs);
- change of numeraire and change of measure;
- linear interest-rate derivatives: FRAs, futures, and swaps;
- bootstrapping and interpolation of the interest-rate curve;
- interest-rate options: bond options, caps and floors, and swaptions;
- market models, including Black and Bachelier pricing formulas;
- short-rate models, including Vasicek, CIR, and the one-factor Hull–White model;

- an overview of Heath–Jarrow–Morton models, an introduction to RFR modelling when time permits, and interest-rate-risk hedging.

Pricing and modelling are developed primarily in a single-curve framework. The distinction between discounting and forwarding curves and the main features of RFR-based instruments are introduced; multicurve and further RFR extensions are discussed as time permits.

A.1 Computational applications and homework

Guided Python examples and homework cover selected applications: curve bootstrapping and interpolation; coupon-bond and floating-rate-note pricing; transformation of cap and swaption volatility quotes into Black/Bachelier market prices; and Hull–White calibration, option pricing, and Monte Carlo simulation.

A.2 Assessment

The maximum grade is 30/30 cum laude. The written examination accounts for 75% and assesses theoretical foundations plus analytical and computational problem solving. A compulsory computational group project with an individual oral defence accounts for 25%. The defence checks understanding, individual contribution, and the ability to interpret and critically discuss the results. Detailed criteria and deadlines are published on Moodle. The supplied syllabus lists five sessions (December, January, February, June, and September) and permits up to three examination attempts during the academic year.

Outstanding project work may be taken into account in determining the final grade and, where appropriate, in the award of cum laude. Participation in the winter examination session is strongly encouraged.

A.3 References

- D. Brigo and F. Mercurio, *Interest Rate Models: Theory and Practice With Smile, Inflation and Credit*, Springer, 2006.
- M. Henrard, *Interest Rate Modelling in the Multi-Curve Framework: Foundations, Evolution, Transition, and Implementation*, 2nd ed., Springer, 2026.
- T. Björk, *Arbitrage Theory in Continuous Time*, Oxford University Press, 2009.
- J. Kienitz, *Interest Rates Explained*, vols. 1–2, Palgrave, 2013 and 2017.